

数据结构

Data Structure

7

图

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 图的定义和术语
- 图的存储结构
- 图的遍历
- 最小生成树
- 有向无环图及其应用
- 最短路径

目录

1	图的定义和术语
2	图的存储结构
3	图的遍历
4	最小生成树
5	有向无环图及其应用

图的定义和术语

- 图（Graph）是一种多对多的非线性数据结构。

- 树有根，链表有头尾

- 图研究的核心是顶点之间的连通关系。

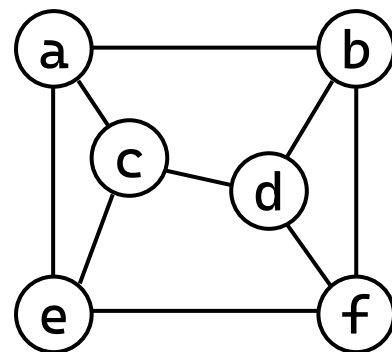
- 图的数据元素是顶点。如：a, b。

- 图的逻辑结构是关系。如： $\langle a, b \rangle$ 。

- 边（Edge）：指的是两个顶点之间直接的连接。

- 如果 $\langle v, w \rangle \in VR$ 必有 $\langle w, v \rangle \in VR$ ，则称 (v, w) 是 v 和 w 之间的一条边

- 边没有次序关系，即 (v, w) 和 (w, v) 表示同一条边。



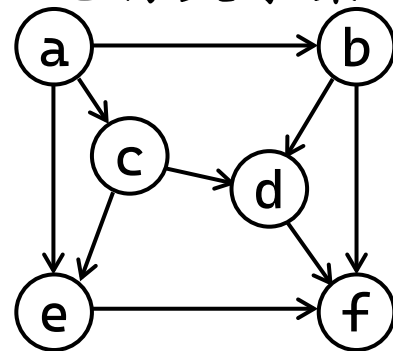
设A和B是两个集合。 $A \times B$ 的任意一个子集R，称为从A到B的一个二元关系。

- 如果 $A=B$ ，则称 R 为 A 上的二元关系。

- 若有序对 $\langle x, y \rangle \in R$ ，则称 x 与 y 具有关系 R。

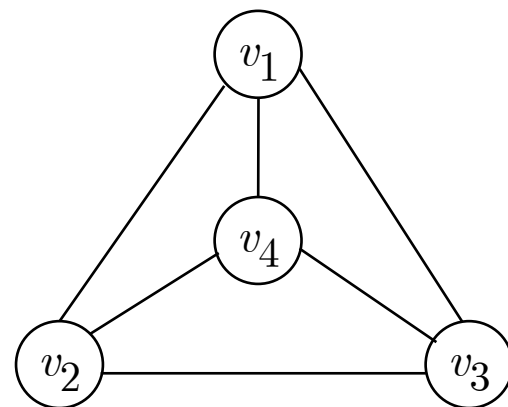
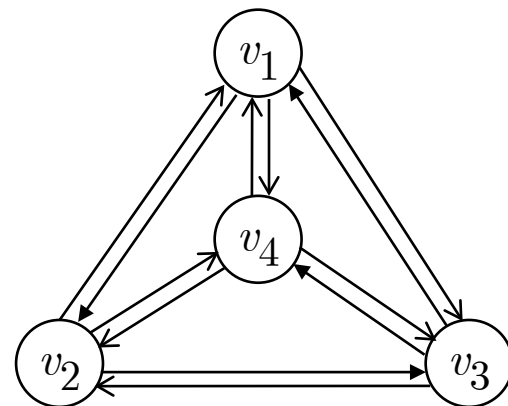
图的定义和术语

- 图 G 是由两个集合组成的二元组 $G = (V, E)$ 。
 - V (Vertex Set) : 顶点的非空有限集。如 $V = \{a, b, c, d, e, f\}$ 。
 - E (Edge/Arc Set) : 顶点之间关系的有限集，也称关系集。
 - 是 $E \subseteq V \times V$ 的一个子集。
 - 如 : $E = VR = \{ \langle a, b \rangle, \langle b, c \rangle, \langle c, a \rangle, \langle d, f \rangle, \dots \}$
- 无向图 : 由顶点集和边集构成的图。
- 有向图 : 由顶点集和弧集 (有次序关系) 构成的图。
 - 弧 (Arc) : 如果 $\langle v, w \rangle \in VR$, 则 $\langle v, w \rangle$ 表示从 v 到 w 的一条弧。
 - 弧头 (Head) : 箭头端顶点 ; 弧尾 (Tail) : 箭头另一顶点。



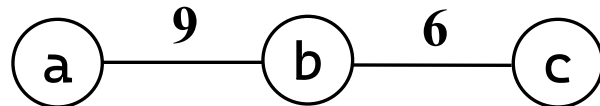
图的定义和术语

- 设 $\langle v_i, v_j \rangle \in VR$, $v_i \neq v_j$; 图中顶点数目 n ; 图中边 (或弧) 的数目 m ; 则:
 - 对于有向图: $M=n(n-1)$;
 - 对于无向图: $M=n(n-1)/2$ 。
 - 对有向图和无向图有 $0 \leq m \leq M$ 。
- 完全图 等定义
 - 完全图: $m=M$ 。
 - 稀疏图: $m \ll M$ 。
 - 稠密图: m 接近于 M 。



图的定义和术语

- 权值 (Weight) : 为图中的边或弧所赋予的一个数值 , 用于量化其某种特性或成本。

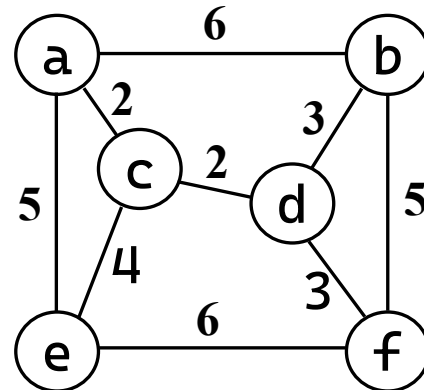
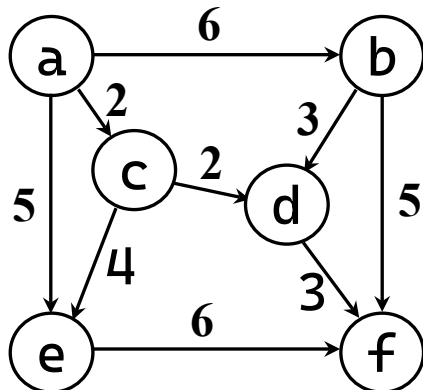


- 网 (Network) : 指带有权值的图。

– 网是边集 E 上定义了一个权值函数 $W:E \rightarrow R$ 的图 $G=(V, E)$ 。

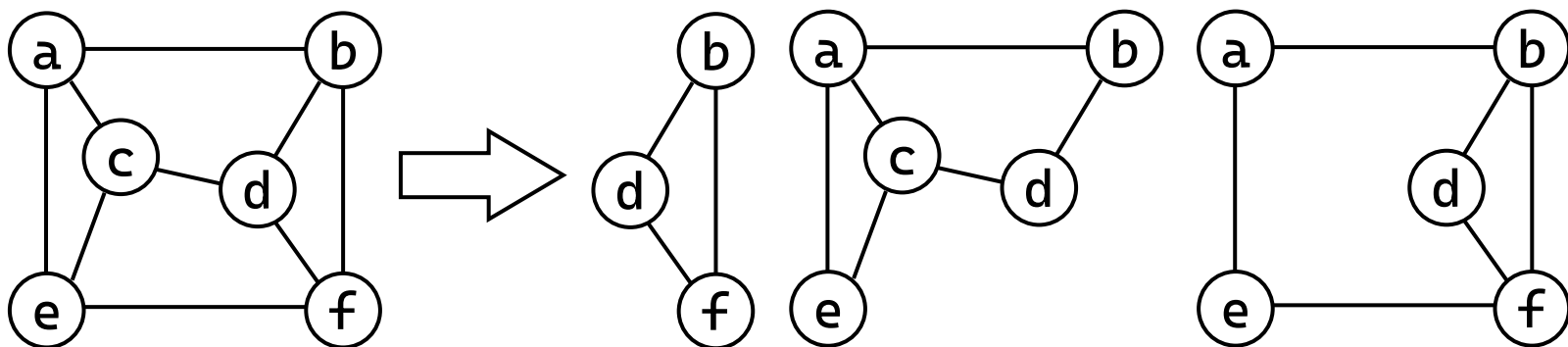
– 在不会引起歧义时，网也直接称为图。

– 有向网：带权的有向图；无向网：带权的无向图。



图的定义和术语

- 子图：设图 $G=(V, E)$ ， $G'=(V', E')$ ，若 $V' \subseteq V$ 且 $E' \subseteq E$ ，则称 G' 为 G 的子图。

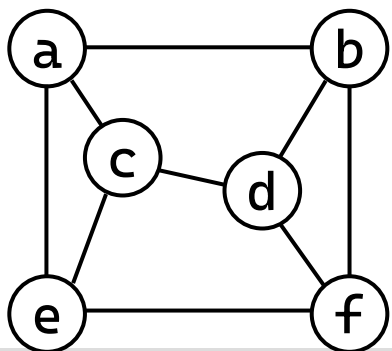


- 邻接点 (Adjacent)

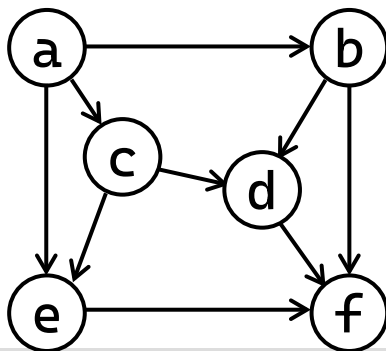
- 对于有向图 $G=(V, E)$ ，如果边 $(v, v') \in E$ ，则称顶点 v 和 v' 互为邻接点(即相邻接)，边 (v, v') 与 v 和 v' 相关联。
- 对于无向图 $G=(V, E)$ ，如果弧 $\langle v, v' \rangle \in E$ ，则称 v 邻接到 v' ， v' 邻接自 v ；弧 $\langle v, v' \rangle$ 和顶点 v, v' 相关联。

图的定义和术语

- 顶点的度：与顶点 v 相邻接的边的数目，记为 $TD(v)$ 。
 - 有向图顶点 v 的入度 $ID(v)$ ：以 v 为弧头的弧的数目
 - 有向图顶点 v 的出度 $OD(v)$ ：以 v 为弧尾的弧的数目
 - 有向图顶点 v 的度： $TD(v) = ID(v) + OD(v)$ 。
 - 无向图顶点 v 的度：记为 $TD(v)$ 。
 - 边长 $m = \frac{1}{2} \sum_{i=1}^n TD(v_i)$



$$TD(c) = 3$$



$$ID(c) = 1$$

$$OD(c) = 2$$

$$TD(c) = ID(c) + OD(c) = 3$$

图的定义和术语

• 路径

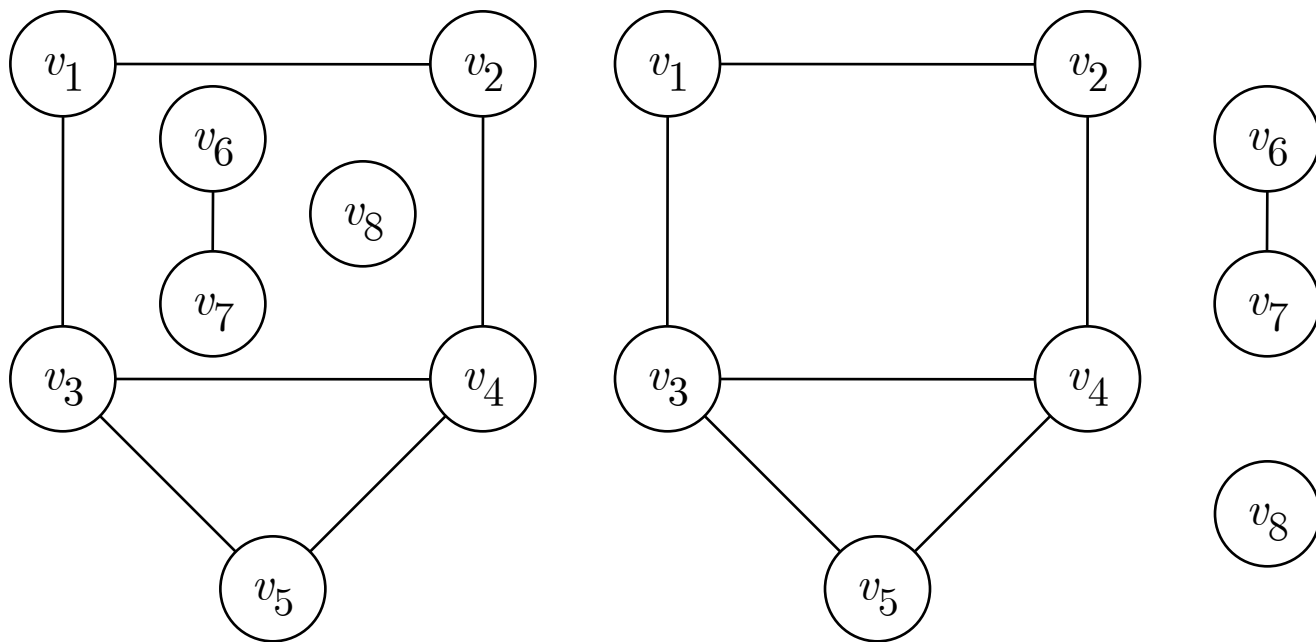
- 设图 $G=(V, E)$ 为一个无向图或有向图，若存在一个顶点序列 $v=v_0, v_1, \dots, v_k=v'$ ，满足：
- 边的存在性：对于任意 $j \in \{1, 2, \dots, k\}$ ，顶点对 $(v_{j-1}, v_j) \in E$ 。
 - 方向性：在无向图中，边 (v_{j-1}, v_j) 无方向；在有向图中，边必须为有向边 $\langle v_{j-1}, v_j \rangle$ 且方向从 v_{j-1} 指向 v_j 。
- 起点与终点：序列首顶点 v_0 为路径的起点，末顶点 v_k 为终点。
- 路径的长度：路径中边的数量（或跳数）为 $|P| = k$ 。
- 路径的构成：路径 P 可以表示为顶点序列 $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ ，或简记为 $P = \langle v_0, v_1, \dots, v_k \rangle$ 。

图的定义和术语

- 简单路径与回路
 - 简单路径：路径中所有顶点各不相同（即 $\forall i \neq j, v_i \neq v_j$ ）。
 - 回路（Cycle）或环：起点与终点相同（ $v_0 = v_k$ ）
 - 简单回路：简单路径中的回路。
- v 和 v' 连通： v 和 v' 之间有路径。
- 连通图：无向图中任意两个顶点都连通。
 - 强连通图：有向图中， $\forall i \neq j, v_i$ 到 v_j 、 v_j 到 v_i 都有路径。
- 连通分量：无向图中的极大连通子图。
 - 强连通分量：有向图中的极大连通子图。

图的定义和术语

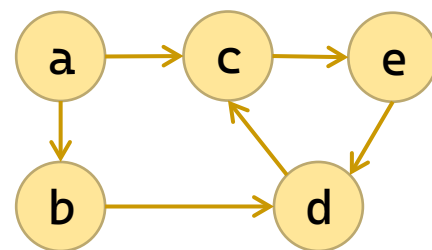
- 例如，非连通图及其3个连通分量。



图的定义和术语

- 例 设图 $G=(V, E)$, $V=\{a, b, c, d, e\}$,
 $E=\{ \langle a, b \rangle, \langle a, c \rangle, \langle b, d \rangle, \langle c, e \rangle, \langle d, c \rangle, \langle e, d \rangle \}$

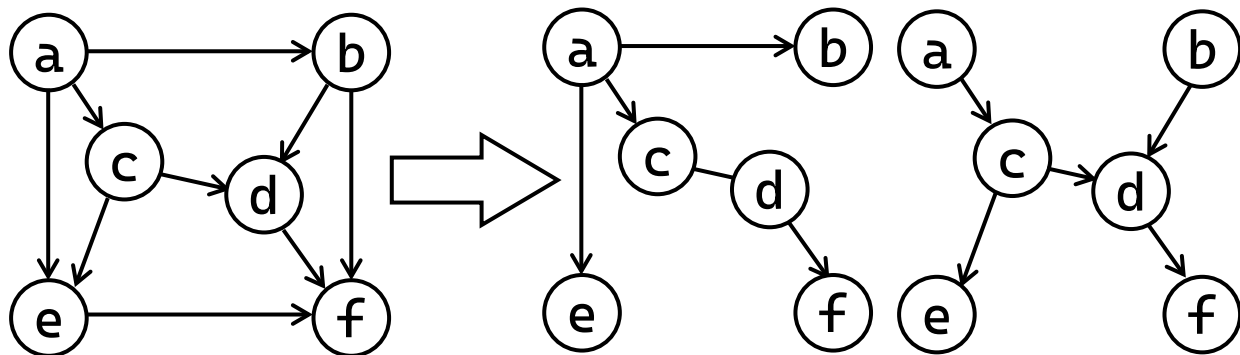
- 回答下列问题：



- (1) 求与顶点b相关联的弧； $\langle a, b \rangle, \langle b, d \rangle$
- (2) 是否存在从c到b的路径？ 不存在， $\langle c, e \rangle, \langle e, d \rangle, \langle d, c \rangle$
- (3) 求ID(d)、OD(d)、TD(d) ; ID(d)=2、OD(d)=1、TD(d)=3
- (4) 该有向图是否是强连通图？ 不是，不存在c到b的路径。
- (5) 画出各个强连通分量。 ① $\langle a \rangle$; ② $\langle b \rangle$;
③ $\langle c, e \rangle, \langle e, d \rangle, \langle d, c \rangle$ 。

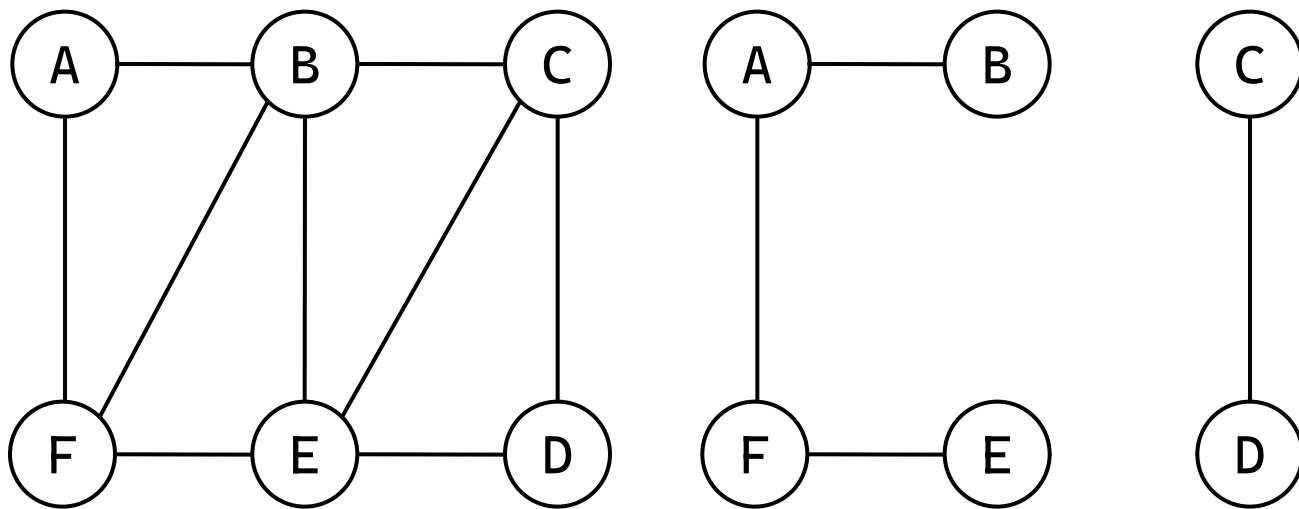
图的定义和术语

- 生成树：设连通图 G 共有 n 个顶点，则含有图 G 的 n 个顶点且有 $n-1$ 条边的连通图，称为图 G 的一棵生成树。
 - 有 n 个顶点和 $n-1$ 条边的图不一定是生成树。
 - 在有 n 个顶点的图中，如果边多于 $n-1$ 条，则一定有环；如果边少于 $n-1$ 条，则是非连通图。
- 有向树：只有一个顶点的入度为 0，其它顶点的入度都为 1 的有向图。



图的定义和术语

- 生成森林：含有图中全部 n 个顶点的 m 棵互不相交的有向树(共有 $n-m$ 条弧)。

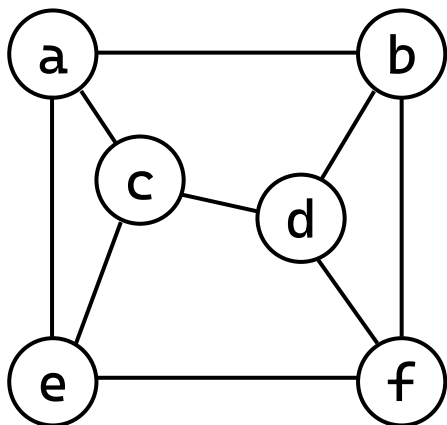


目录

1	图的定义和术语
2	图的存储结构
3	图的遍历
4	最小生成树
5	有向无环图及其应用

图的邻接矩阵

- 例 对于无向图

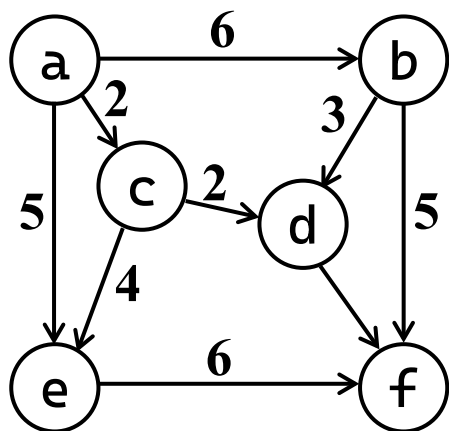


$V_e = \{ a, b, c, d, e, f \}$

Vr	a	b	c	d	e	f
a	0	1	1	0	1	0
b	1	0	0	1	0	1
c	1	0	0	1	1	0
d	0	1	1	0	0	1
e	1	0	1	0	0	1
f	0	1	0	1	1	0

图的邻接矩阵

- 例 对于有向网



$V_e = \{ a, b, c, d, e, f \}$

V_r	a	b	c	d	e	f
a	0	6	2	0	5	0
b	0	0	0	3	0	5
c	0	0	0	2	4	0
d	0	0	0	0	0	3
e	0	0	0	0	0	6
f	0	0	0	0	0	0

图的邻接矩阵

- 邻接矩阵，也称为数组表示法
 - 对于无权图：adj取0表示无边/不邻接，取1表示有边/邻接。
 - 对于带权图（网）：adj直接存储权值（如距离、花费）。此时，通常用0或MAX表示“不连通”。

```
typedef struct {  
    int adj;  
} ArcNode;  
typedef struct AdjMatrix {  
    char vertex[MAX_VERTEX];  
    ArcNode arcs[MAX_VERTEX][MAX_VERTEX];  
    int vexnum;  
} AdjMatrix;
```

图的邻接矩阵：初始化和定位

- 初始化：将邻接矩阵置0或者INF

```
void init_graph(AdjMatrix* G) {  
    G->vexnum = 0;  
    for (int i = 0; i < MAX_VERTEX; i++)  
        for (int j = 0; j < MAX_VERTEX; j++)  
            G->arcs[i][j].adj = INFINITY;  
}
```

- 定位顶点：根据顶点名字查询顶点下表

```
int locate_vertex(AdjMatrix* G, char v) {  
    for (int i = 0; i < G->vexnum; i++)  
        if (G->vertex[i] == v)  
            return i;  
    return ERROR;  
}
```

图的邻接矩阵：插入顶点

- 插入顶点：新建顶点，与其它顶点的权重置为最大

```
int insert_vertex(AdjMatrix* G, char v) {  
    if (locate_vertex(G, v) != ERROR)    // 已存在  
        return ERROR;  
    if (G->vexnum >= MAX_VERTEX)        // 容量不足  
        return ERROR;  
    G->vertex[G->vexnum] = v;  
    // 新顶点与其他顶点的连接初始化为 INFINITY  
    for (int i = 0; i <= G->vexnum; i++) {  
        G->arcs[i][G->vexnum].adj = INFINITY;  
        G->arcs[G->vexnum][i].adj = INFINITY;  
    }  
    G->vexnum++;  
    return TRUE;  
}
```

图的邻接矩阵：删除顶点

- 删除顶点

- 删除出边：后面的边（行）依次往前覆盖

```
for (int i = loc; i < G->vexnum - 1; i++)  
    for (int j = 0; j < G->vexnum; j++)  
        G->arcs[i][j].adj = G->arcs[i + 1][j].adj;
```

- 删除入边：后面的边（列）依次往前覆盖

```
for (int i = loc; i < G->vexnum - 1; i++) {  
    for (int j = 0; j < G->vexnum; j++) {  
        G->arcs[j][i].adj = G->arcs[j][i + 1].adj;
```

- 删除顶点：后面的顶点依次往前覆盖

```
for (int i = loc; i < G->vexnum - 1; i++)  
    G->vertex[i] = G->vertex[i + 1];
```

- 顶点长度收缩

```
G->vexnum--;
```

```

int delete_vertex(AdjMatrix* G, char v) {
    int loc = locate_vertex(G, v);
    if (loc == ERROR)
        return ERROR;
    // 行上移 (将后面的行覆盖到前面)
    for (int i = loc; i < G->vexnum - 1; i++)
        for (int j = 0; j < G->vexnum; j++)
            G->arcs[i][j].adj = G->arcs[i + 1][j].adj;
    // 列左移 (将后面的列覆盖到前面)
    for (int i = loc; i < G->vexnum - 1; i++)
        for (int j = 0; j < G->vexnum; j++)
            G->arcs[j][i].adj = G->arcs[j][i + 1].adj;
    // 顶点表前移
    for (int i = loc; i < G->vexnum - 1; i++)
        G->vertex[i] = G->vertex[i + 1];
    G->vexnum--;
    return TRUE;
}

```

图的邻接矩阵：插入弧

- 插入弧：找到点，更新权重

```
int insert_arc(AdjMatrix* G, char vi, char vt, int weight) {  
    int row = locate_vertex(G, vi);  
    int col = locate_vertex(G, vt);  
    if (row == ERROR || col == ERROR)  
        return ERROR;  
    G->arcs[row][col].adj = weight;  
    return TRUE;  
}
```

图的邻接矩阵：删除弧

- 删除弧：找到点，更新权重为无限大

```
int delete_arc(AdjMatrix* G, char vi, char vt) {  
    int row = locate_vertex(G, vi);  
    int col = locate_vertex(G, vt);  
    if (row == ERROR || col == ERROR  
        || G->arcs[row][col].adj == INFINITY)  
        return ERROR;  
    G->arcs[row][col].adj = INFINITY;  
    return TRUE;  
}
```

图的邻接矩阵：创建图

• 创建图

— 初始化

```
init_graph(G);
```

— 循环接受输入点

```
for (i = 0; input[i]!='\0' && i<MAX_VERTEX; i++)
```

— 插入点

```
insert_vertex(G, input[i])
```

— 初始化对角线

```
for (int i = 0; i < G->vexnum; i++) {  
    G->arcs[i][i].adj = 0;
```

— 循环接受输入弧

```
while (1) {  
    scanf(" %c %c %d", &v1, &v2, &weight);
```

— 插入弧

```
insert_arc(G, v1, v2, weight);
```

```
}
```

```

void create_graph(AdjMatrix* G) {
    char input[MAX_VERTEX + 1];    // 存放输入的顶点字符串
    char v1, v2;
    int weight;
    init_graph(G);
    printf("请在一行内输入所有顶点 (如 'ABCDE') : ");
    scanf("%s", input);
    getchar();    // 吸收换行
    // 将输入的每个字符作为一个顶点插入
    for (int i = 0; input[i] != '\0' && i < MAX_VERTEX; i++)
        if (insert_vertex(G, input[i]) == ERROR)
            printf("警告: 顶点 %c 插入失败.\n", input[i]);
    // 初始化矩阵: 所有元素为 INFINITY, 对角线为 0
    for (int i = 0; i < G->vexnum; i++)
        G->arcs[i][i].adj = 0;
    printf("请输入弧 (格式: 起点 终点 权重) , 输入 '#' 结束: \n");
    while (1) {
        printf("> ");
        scanf(" %c", &v1);
    }
}

```

```

    if (v1 == '#')
        break;
    scanf("%c %d", &v2, &weight);
    int result = insert_arc(G, v1, v2, weight);
    if (result == ERROR)
        printf("插入弧 %c->%c 失败 (顶点不存在) \n", v1, v2);
    else
        printf("插入成功\n");
}
printf("弧输入结束.\n");
}

```

图的邻接矩阵：计算出入度

• 计算出入度

```
void print_all_degrees(AdjMatrix* G) {
    printf("顶点\t出度\t入度\t度\n");
    for (int i = 0; i < G->vexnum; i++) {
        char v = G->vertex[i];
        int out_deg = 0, in_deg = 0;
        for (int j = 0; j < G->vexnum; j++) {
            if (G->arcs[i][j].adj!=INFINITY && G->arcs[i][j].adj!=0)
                out_deg++;
            if (G->arcs[j][i].adj!=INFINITY && G->arcs[j][i].adj!=0)
                in_deg++;
        }
        printf(" %c\t%d\t%d\t%d\n", v, out_deg, in_deg, out_deg+in_deg);
    }
    printf("=====\n");
}
```

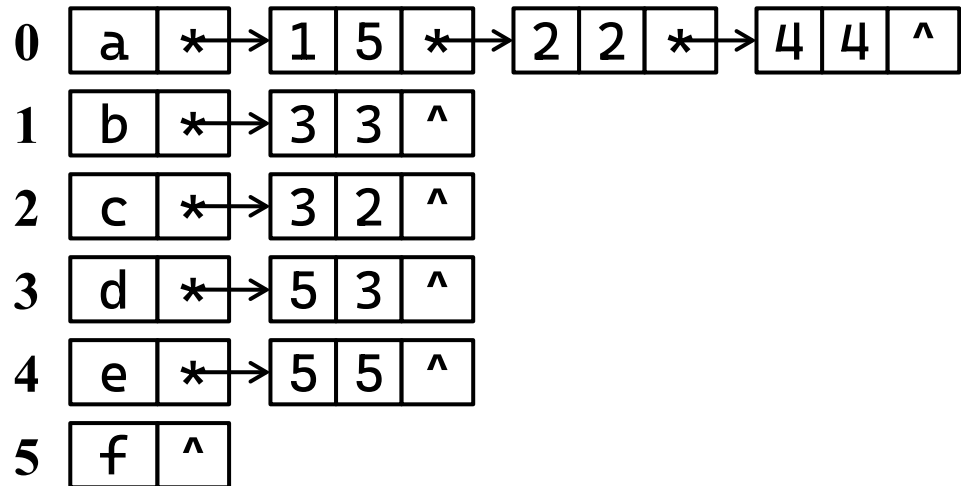
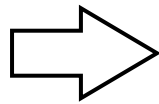
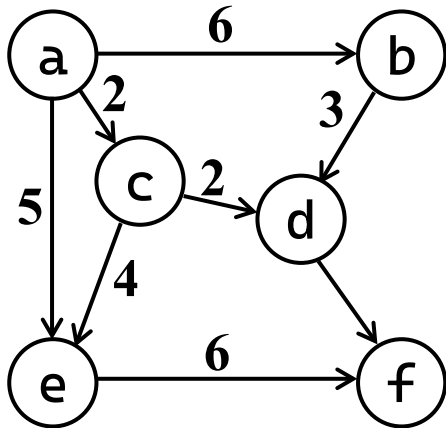
图的邻接表

- 图的邻接表存储表示：

- 图的一种链式存储结构；

- 图中每一个顶点对应1个结点，并由所有顶点结点构成一个顺序表(结构)；

- 图中每一个顶点建立一条链，该链由它的所有邻接点构成。



图的邻接表

- 无向图邻接表

- 1条边对应2个表结点，即结点总数是边数的2倍；
- 顶点的度=邻接点链表中结点的数目。

- 有向图邻接表

- 1条弧对应1个表结点，即结点总数=弧的数目；
- 顶点的出度=邻接点链表中结点数目。
- 在有向图邻接表中，求顶点的入度比较复杂，需搜索所有的邻接点链表后才能确定。
- 对有向图建立逆邻接链表，则顶点的入度=逆邻接点链表中结点的数目。

图的邻接表

• 图的邻接表的存储结构

```
typedef struct ArcNode {           // 边结点 (链表结点)
    int adjvex;                    // 邻接点下标
    int weight;                    // 权值
    struct ArcNode* next;         // 下一条边
} ArcNode;
typedef struct {                  // 顶点结点
    char data;                    // 顶点数据
    ArcNode* firstarc;            // 第一条边
} VertexNode;
typedef struct {                  // 邻接表表示的图
    VertexNode vertex[MAX_VERTEX];
    int vexnum;                  // 顶点数
} AdjList;
```

图的邻接表：初始化和定位

- 初始化：顶点数归零，所有顶点的边置空

```
void init_graph(AdjList* G) {  
    G->vexnum = 0;  
    for (int i = 0; i < MAX_VERTEX; i++) {  
        G->vertex[i].firstarc = NULL;  
    }  
}
```

- 定位顶点：根据顶点名字查询顶点下表

```
int locate_vertex(AdjList* G, char v) {  
    for (int i = 0; i < G->vexnum; i++)  
        if (G->vertex[i].data == v)  
            return i;  
    return ERROR;  
}
```

图的邻接表：插入顶点

- 插入顶点：新建顶点，该顶点的边置空

```
int insert_vertex(AdjList* G, char v) {  
    if (locate_vertex(G, v) != ERROR)  
        return ERROR;  
    if (G->vexnum >= MAX_VERTEX)  
        return ERROR;  
  
    G->vertex[G->vexnum].data = v;  
    G->vertex[G->vexnum].firstarc = NULL;  
    G->vexnum++;  
    return OK;  
}
```

图的邻接表：删除顶点

• 删除顶点

— 删除出边：顶点下所有边

— 顶点下的边置空

— 删除入边：循环各点

▪ 从首边开始

▪ 找到当前边的前结点

▪ 删除当前结点

```
ArcNode* p = G->vertex[j].firstarc;  
while (p != NULL) {  
    ArcNode* q = p;  
    p = p->next;  
    free(q);  
}
```

```
G->vertex[j].firstarc = NULL;
```

```
for (int i = 0; i < G->vexnum; i++) {
```

```
ArcNode* pre=NULL, * cur=G->vertex[i].firstarc;
```

```
while (cur != NULL) {  
    if (cur->adjvex == j) {  
        if (pre == NULL)  
            G->vertex[i].firstarc=cur->next;  
        else  
            pre->next=cur->next;  
        free(cur);  
        break;    // 不会有重复边  
    }  
    pre = cur; cur = cur->next;  
}
```

图的邻接表：删除顶点

- 删除顶点

- 删除顶点：

- 后面的顶点

- 依次往前覆盖

- 顶点长度收缩

```
for (int i = j; i < G->vexnum - 1; i++) {  
    G->vertex[i].data = G->vertex[i + 1].data;  
    G->vertex[i].firstarc = G->vertex[i + 1].firstarc;  
}
```

```
G->vexnum--;
```

```

int delete_vertex(AdjList* G, char v) {
    int j = locate_vertex(G, v);
    if (j == ERROR)
        return ERROR;
    // 1. 删除所有以 v 为起点的边 (出边)
    ArcNode* p = G->vertex[j].firstarc;
    while (p != NULL) {
        ArcNode* q = p;
        p = p->next;
        free(q);
    }
    G->vertex[j].firstarc = NULL;
    // 2. 删除所有指向 v 的边 (入边) : 遍历所有顶点, 删除其链表中
    //    adjvex == j 的结点
    for (int i = 0; i < G->vexnum; i++) {
        ArcNode* pre = NULL, * cur = G->vertex[i].firstarc;
        while (cur != NULL) {
            if (cur->adjvex == j) {
                if (pre == NULL)
                    G->vertex[i].firstarc = cur->next;
                else
                    pre->next = cur->next;
                free(cur);
                break; // 无重边
            }
            pre = cur;
            cur = cur->next;
        }
    }
}

```

```

        pre = cur;
        cur = cur->next;
    }
}

// 3. 将顶点表中后面的顶点前移, 并更新所有邻接点下标 (>j 的减 1)
// 先将顶点数据前移
for (int i = j; i < G->vexnum - 1; i++) {
    G->vertex[i].data = G->vertex[i + 1].data;
    G->vertex[i].firstarc = G->vertex[i + 1].firstarc;
}
G->vexnum--;

// 4. 更新所有链表中 adjvex > j 的结点, 将其减 1
for (int i = 0; i < G->vexnum; i++) {
    ArcNode* cur = G->vertex[i].firstarc;
    while (cur != NULL) {
        if (cur->adjvex > j)
            cur->adjvex--;
        cur = cur->next;
    }
}
return OK;
}

```

图的邻接表：插入弧

- 插入弧

- 找到插入位置

- 邻接点升序

```
ArcNode* pre=NULL, * cur=G->vertex[i].firstarc;  
while (cur != NULL && cur->adjvex < j) {  
    pre = cur;  
    cur = cur->next;  
}
```

- 如果已有则更新

```
if (cur != NULL && cur->adjvex == j) {  
    cur->weight = weight;  
    return OK;  
}
```

- 新建边

```
ArcNode* p = (ArcNode*)malloc(sizeof(ArcNode));  
p->adjvex = j; p->weight = weight; p->next = cur;
```

- 接上

- 判断首结点

```
if (pre == NULL)  
    G->vertex[i].firstarc = p;  
else  
    pre->next = p;
```

```

int insert_arc(AdjList* G, char vi, char vt, int weight) {
    int i = locate_vertex(G, vi);
    int j = locate_vertex(G, vt);
    if (i == ERROR || j == ERROR)
        return ERROR;
    ArcNode* pre = NULL, * cur = G->vertex[i].firstarc;
    while (cur != NULL && cur->adjvex < j) { // 查找前结点
        pre = cur;
        cur = cur->next;
    }
    if (cur != NULL && cur->adjvex == j) { // 已存在: 更新权重
        cur->weight = weight;
        return OK;
    }
    // 不存在: 创建新结点插入
    ArcNode* p = (ArcNode*)malloc(sizeof(ArcNode));
    p->adjvex = j; p->weight = weight; p->next = cur;
    if (pre == NULL)
        G->vertex[i].firstarc = p;
    else
        pre->next = p;
    return OK;
}

```

图的邻接表：删除弧

- 删除弧

- 找到前一位置

```
ArcNode* pre=NULL, * cur=G->vertex[i].firstarc;  
while (cur != NULL && cur->adjvex != j) {  
    pre = cur;  
    cur = cur->next;  
}  
if (cur == NULL) return ERROR;
```

- 删除边

- 判断首结点

```
if (pre == NULL)  
    G->vertex[i].firstarc = cur->next;  
else  
    pre->next = cur->next;  
free(cur);
```

```

int delete_arc(AdjList* G, char vi, char vt) {
    int i = locate_vertex(G, vi);
    int j = locate_vertex(G, vt);
    if (i == ERROR || j == ERROR)
        return ERROR;
    ArcNode* pre = NULL, * cur = G->vertex[i].firstarc;
    while (cur != NULL && cur->adjvex != j) {
        pre = cur;
        cur = cur->next;
    }
    if (cur == NULL)    // 未找到
        return ERROR;

    if (pre == NULL)
        G->vertex[i].firstarc = cur->next;
    else
        pre->next = cur->next;
    free(cur);
    return OK;
}

```

图的邻接表：创建图

• 创建图

— 初始化

```
init_graph(G);
```

— 循环接受输入点

```
for (i = 0; input[i]!='\0' && i<MAX_VERTEX; i++)
```

— 插入点

```
insert_vertex(G, input[i])
```

— 循环接受输入弧

```
while (1) {  
    scanf(" %c %c %d", &v1, &v2, &weight);
```

— 插入弧

```
insert_arc(G, v1, v2, weight);
```

```
}
```

```

void create_graph(AdjList* G) {
    char input[MAX_VERTEX + 1];
    char vi, vt;
    int weight;
    init_graph(G);
    printf("请在一行内输入所有顶点 (如 'ABCDE') : ");
    scanf("%s", input); getchar();    // 吸收换行
    for (int i = 0; input[i] != '\0' && i < MAX_VERTEX; i++)
        if (insert_vertex(G, input[i]) == ERROR)
            printf("警告: 顶点 %c 插入失败.\n", input[i]);
    printf("请输入弧 (格式: 起点 终点 权重), 输入 '#' 结束: \n");
    while (1) {
        printf("> "); scanf(" %c", &vi);
        if (vi == '#') break;
        scanf("%c %d", &vt, &weight);
        if (insert_arc(G, vi, vt, weight) == ERROR)
            printf("插入弧 %c->%c 失败 (顶点不存在) \n", vi, vt);
        else
            printf("插入成功\n");
    }
    printf("弧输入结束.\n");
}

```

图的邻接表：计算出入度

- 计算出入度

- 出度由顶点的链表算

```
ArcNode* p = G->vertex[i].firstarc;  
while (p != NULL) {  
    out_deg++;  
    p = p->next;  
}
```

- 入度从顶点链表查找

```
for (int i = 0; i < G->vexnum; i++) {  
    p = G->vertex[i].firstarc;  
    while (p != NULL) {  
        if (p->adjvex == loc)  
            in_deg++;  
        p = p->next;  
    }  
}
```

```

void print_all_degrees(AdjMatrix* G) {
    printf("顶点\t出度\t入度\t度\n");
    for (int i = 0; i < G->vexnum; i++) {
        char v = G->vertex[i].data;
        int out_deg = 0, in_deg = 0;
        ArcNode* p = G->vertex[i].firstarc;
        while (p != NULL) {
            out_deg++;
            p = p->next;
        }
        for (int j = 0; j < G->vexnum; j++) {
            p = G->vertex[j].firstarc;
            while (p != NULL) {
                if (p->adjvex == i)
                    in_deg++;
                p = p->next;
            }
        }
        printf(" %c\t%d\t%d\t%d\n", v, out_deg, in_deg,
            out_deg+in_deg);
    }
    printf("=====\n");
}

```

目录

	2	图的存储结构
	3	图的遍历
	4	最小生成树
	5	有向无环图及其应用
	6	最短路径

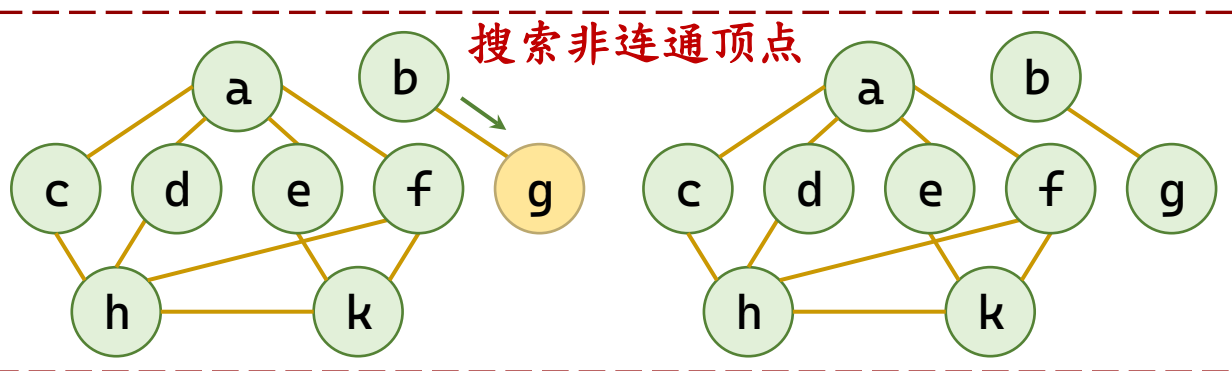
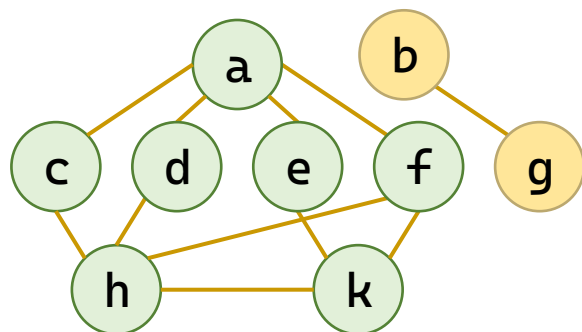
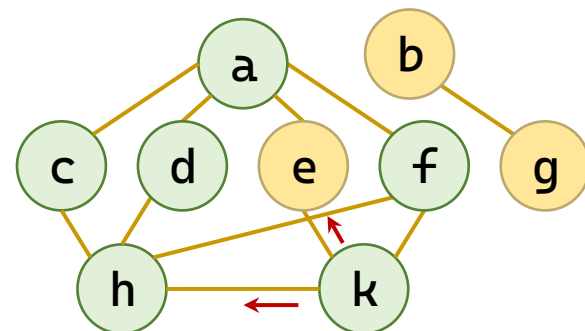
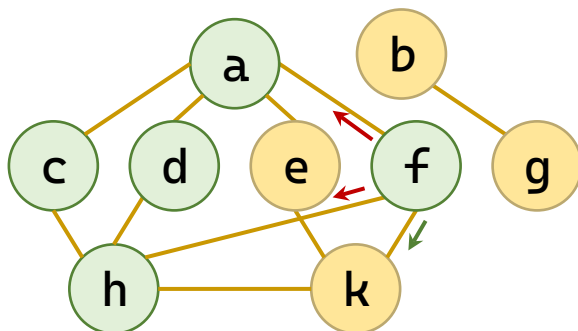
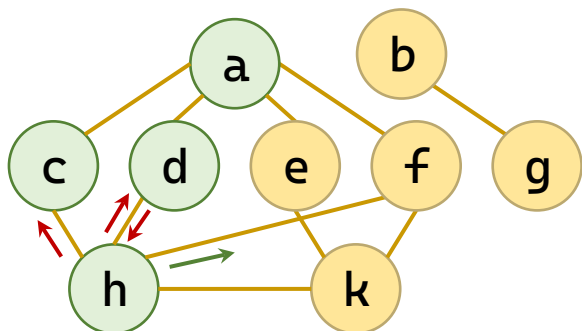
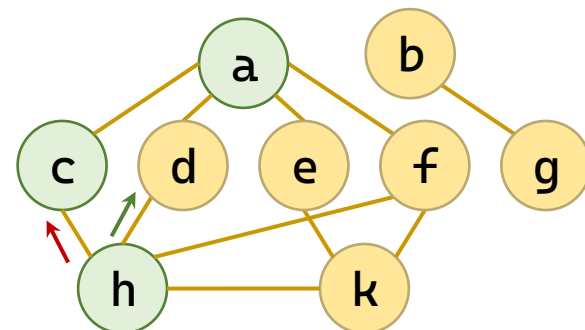
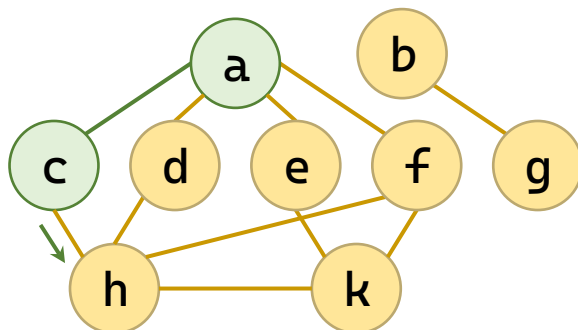
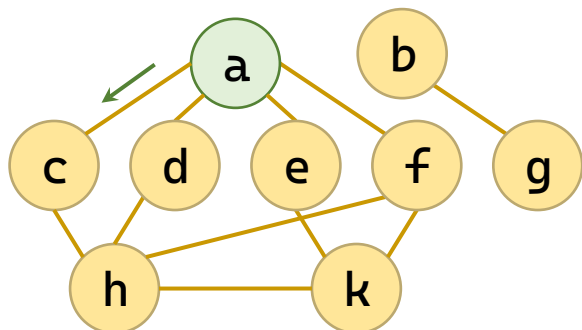
图的遍历

- 从图中某个顶点出发，按照一定规则访问图中的所有顶点，且图中的每个顶点仅被访问一次。
- 基本思想
 - 1、从图中某个顶点 v 出发，访问该顶点；
 - 2、查找顶点 v 的第一个未被访问的邻接点 v_1 ，访问该顶点， $v \leftarrow v_1$ ；
 - 3、重复第2步操作，直到图中没有未被访问的顶点为止。
- 深度优先搜索(Depth First Search)
- 广度优先搜索(Breadth First Search)

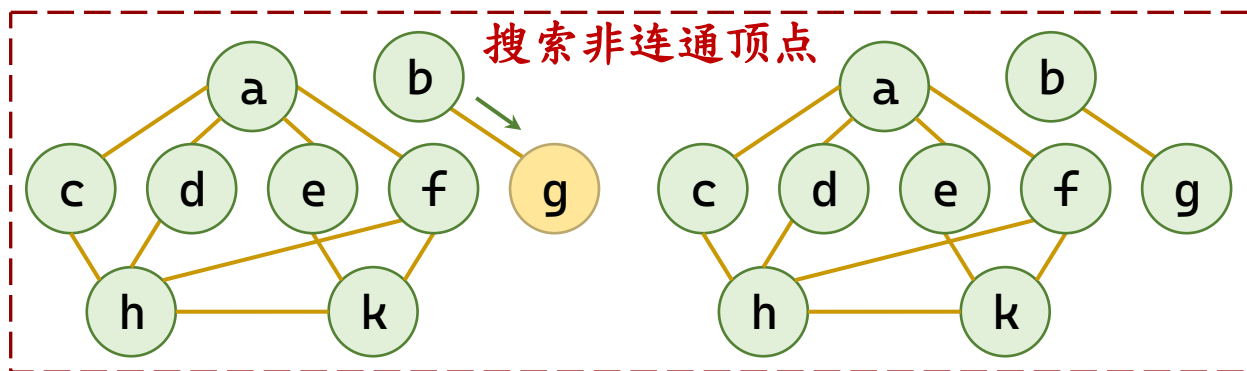
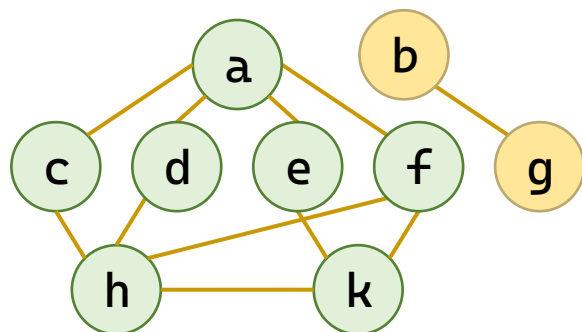
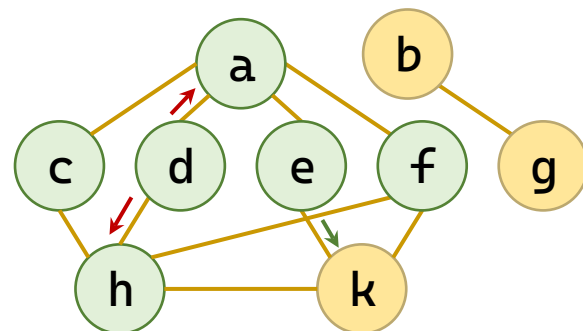
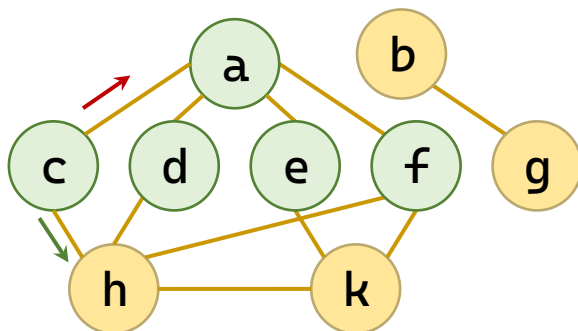
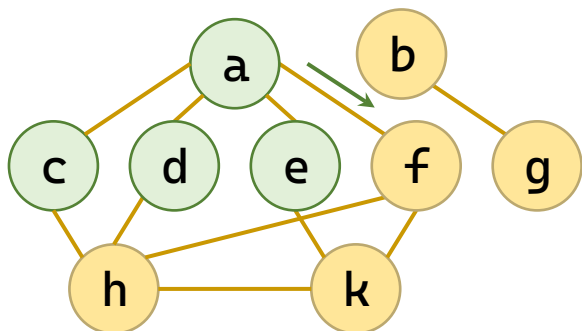
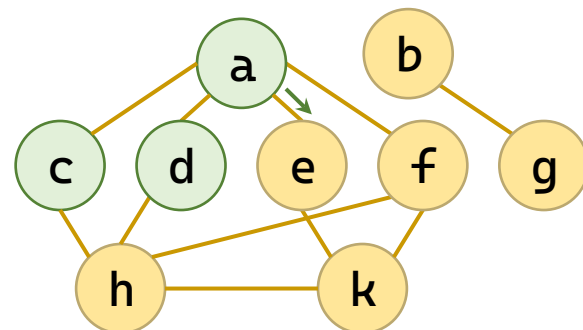
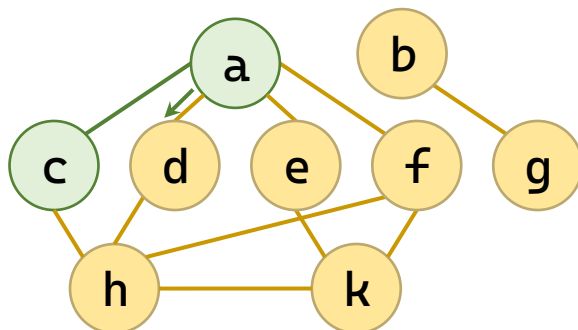
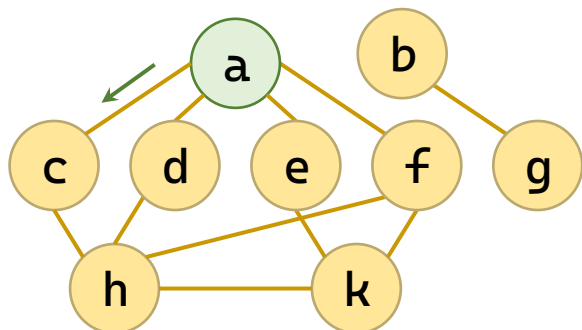
深（广）度优先遍历

- 初始化：防重复数组visited[]清零。
- 启动：遇 $\text{visited}[v]==0$ 的v启动。
- 取出：从栈（DFS）或队列（BFS）中弹出一个顶点。
- 访问：处理该顶点，并将对应visited[v]标记置1。
- 扩展
 - 深度优先：扫描其邻接点，将首个未访问的邻接点压入栈。
 - 广度优先：扫描其邻接点，将所有未访问的邻接点进队列。
- 循环：重复“取出、访问、扩展”直至容器为空。
- 非连通处理：回到“启动”，直至全图标记完毕。

深度优先遍历



广度优先遍历



深度优先和广度优先

- 相同点：取出、访问、扩展
- 不同点：深度优先用栈、广度优先用队列

```
void dfs_stack(int start) {
    int vst[SIZE] = {}; // 已访问标记
    Stack s;
    init_stack(&s);
    push(&s, start);
    vst[start] = 1; // 入栈即标记
    while (!is_stack_empty(&s)) {
        int v = pop(&s);
        printf("%d ", v);
        for (int w = 1; w <= n; w++)
            if (g[v][w]==1 && !vst[w]) {
                v[w] = 1; // 入栈即标记
                push(&s, w);
            }
    }
    printf("\n");
}
```

```
void bfs_queue(int start) {
    int v[SIZE] = {}; // 已访问标记
    Queue q;
    init_queue(&q);
    enqueue(&q, start);
    vst[start] = 1; // 入队即标记
    while (!is_queue_empty(&q)) {
        int v = dequeue(&q);
        printf("%d ", v);
        for (int w = 1; w <= n; w++)
            if (g[v][w]==1 && !vst[w]) {
                v[w] = 1; // 入队即标记
                enqueue(&q, w);
            }
    }
    printf("\n");
}
```

应用：判断两个顶点是否连通

- 例 判断v1和v2是否连通(邻接矩阵存储)

```
bool isConnected(AdjMatrix* G, int b, int c) {
    if (b == c) return true;
    int visited[MAX_VERTEX] = { 0 };    // 访问标示数组
    SeqQueue q; init_queue(&q);
    enqueue(&q, b); visited[b] = 1;    // 起始顶点入队并标记
    while (!is_empty(&q)) {
        int v;
        dequeue(&q, &v);                // 取出：从队首取出一个顶点
        if (v == c) return true;        // 访问：检查是否为目标顶点
        for (int w = 0; w < G->vexnum; w++)    // 扩展：遍历邻接点
            if (G->arcs[v][w].adj != INFINITY && visited[w] == 0) {
                visited[w] = 1; enqueue(&q, w);    // 入队时立即标记
            }
    }
    return false;    // 队列空仍未找到，不连通
}
```

目录

	2	图的存储结构
	3	图的遍历
	4	最小生成树
	5	有向无环图及其应用
	6	最短路径

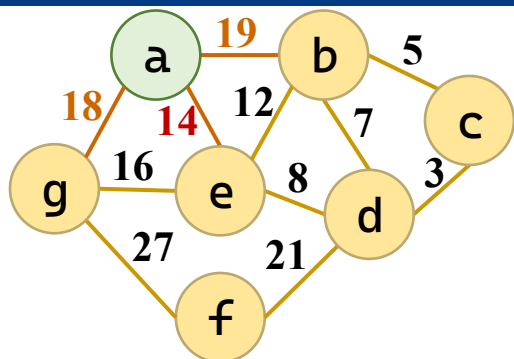
生成树

- 在遍历过程中，所有经过的边记为 T ，没有经过的边(即回边)记为 B 。图 $G'=(V, T)$ 是图 G 的一棵生成树。
 - 深度优先生成树，广度优先生成树
- 最小生成树(Minimum Cost Spanning Tree):
 - 在一个连通网的所有生成树中，代价之和最小的生成树。
 - 例：在 n 个城市间建立一个通信网，修建 $n-1$ 条费用总和最小的通信线路，等价于：构建一棵最小代价生成树。

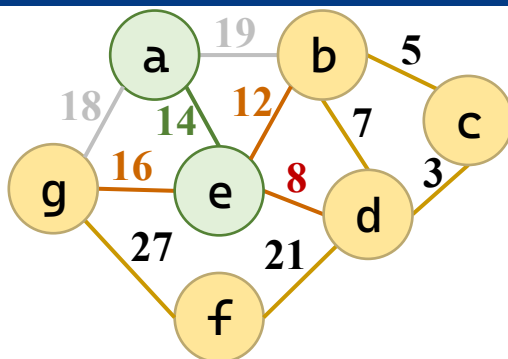
最小生成树

- 最小生成树(Minimum Cost Spanning Tree)
 - 性质：设 $G=(V, E)$ 是一个连通网， U 是顶点集 V 的一个非空子集。若 (u, v) 是一条具有最小权值的边，其中， $u \in U$ ， $v \in V - U$ ，则存在一棵包含边 (u, v) 的最小生成树。
- 方法
 - 算法一：普里姆 (Prim) 算法
 - 算法二：克鲁斯卡尔 (Kruskal) 算法

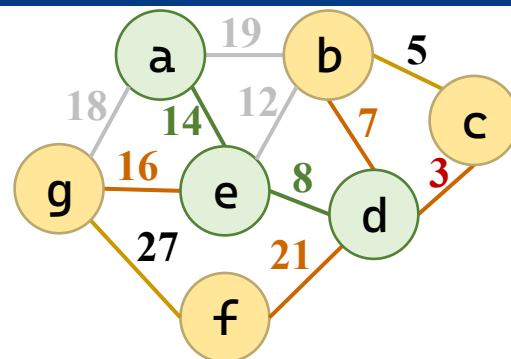
Prim算法——加点法



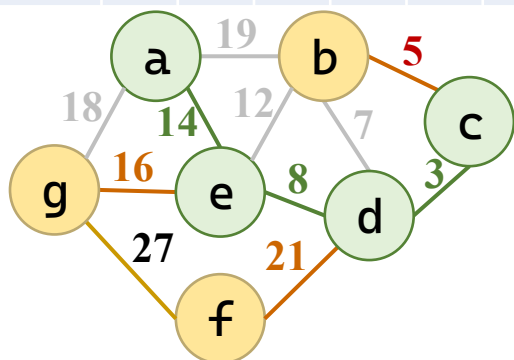
	b	c	d	e	f	g
V_i	a			a		a
代价	19			14		18



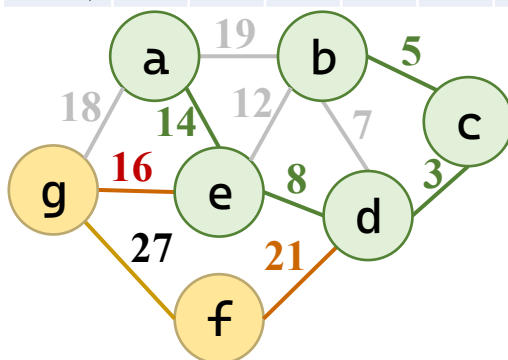
	b	c	d	e	f	g
V_i	e		e	a		e
代价	12		8	14		16



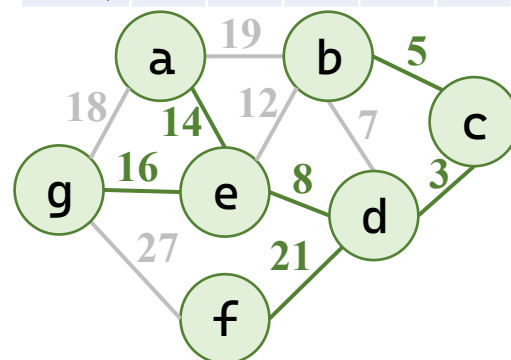
	b	c	d	e	f	g
V_i	d	d	e	a	d	e
代价	7	3	8	14	21	16



	b	c	d	e	f	g
V_i	c	d	e	a	d	e
代价	5	3	8	14	21	16



	b	c	d	e	f	g
V_i	c	d	e	a	d	e
代价	5	3	8	14	21	16



	b	c	d	e	f	g
V_i	c	d	e	a	d	e
代价	5	3	8	14	21	16

Prim算法——加点法

- 算法

- 初始化

```
for (i = 0; i < G->vexnum; i++) {  
    lowcost[i] = G->arcs[start][i].adj;  
    adjvex[i] = start;  
}  
visited[start] = 1; lowcost[start] = 0;
```

- 重复 $N-1$ 次加顶点

```
for (int k = 1; k < G->vexnum; k++) {
```

- 选出最小的未访问顶点

```
for (int i = 0; i < G->vexnum; i++)  
    if (!visited[i] && lowcost[i] < min) {  
        min = lowcost[i]; j = i;  
    }
```

- 选中边

```
mst_edges[mst_count].u = adjvex[j];  
mst_edges[mst_count].v = j;  
mst_edges[mst_count].w = min;  
mst_count++;
```

- 更新代价和
 连接点数组

```
visited[j] = 1;  
for (int i = 0; i < G->vexnum; i++)  
    if (!visited[i] && G->arcs[j][i].adj < lowcost[i]) {  
        lowcost[i] = G->arcs[j][i].adj;  
        adjvex[i] = j;  
    }
```

```

int Prim(AdjMatrix* G, int start) {
    int lowcost[MAX_VERTEX];    // 记录各顶点到当前生成树的最小权值
    int adjvex[MAX_VERTEX];    // 记录该最小权值边对应的树中顶点
    int visited[MAX_VERTEX] = { 0 };    // 标记顶点是否已在生成树中
    // 初始化
    for (int i = 0; i < G->vexnum; i++) {
        lowcost[i] = G->arcs[start][i].adj;
        adjvex[i] = start;
    }
    visited[start] = 1; lowcost[start] = 0;    // 权值为0, 避免选中
    int total_weight = 0, mst_count = 0;
    Edge mst_edges[MAX_VERTEX];    // 存储选中边
    // 循环 n-1 次, 每次加入一个顶点
    for (int k = 1; k < G->vexnum; k++) {
        // 1. 从 lowcost 中选出最小的未访问顶点
        int min = INFINITY;
        int j = -1;
        for (int i = 0; i < G->vexnum; i++)
            if (!visited[i] && lowcost[i] < min) {
                min = lowcost[i];
                j = i;
            }
    }
}

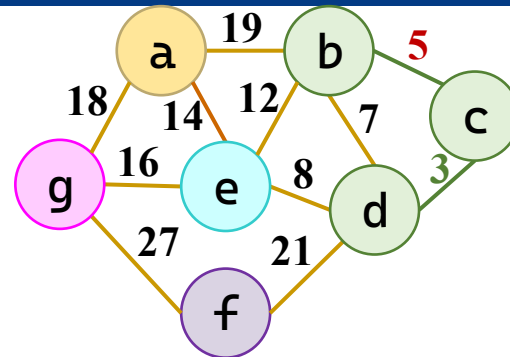
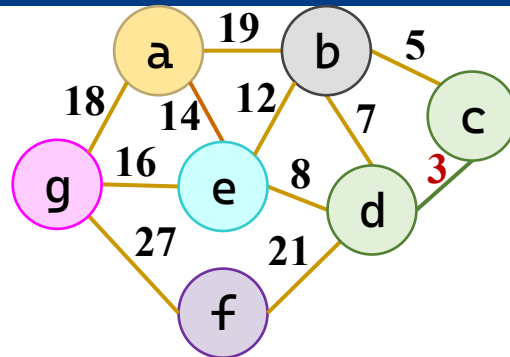
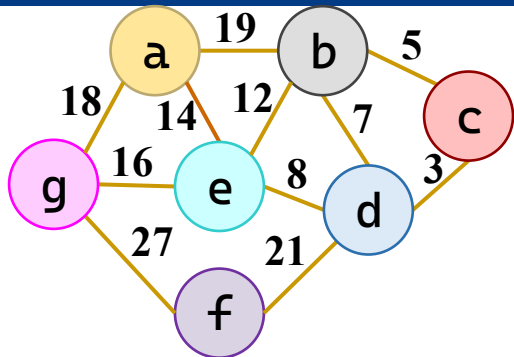
```

```

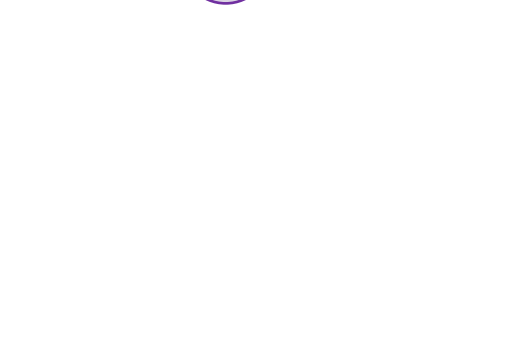
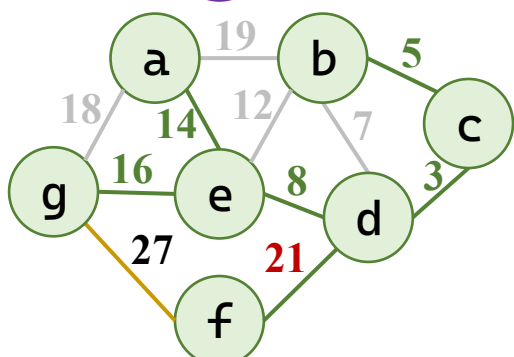
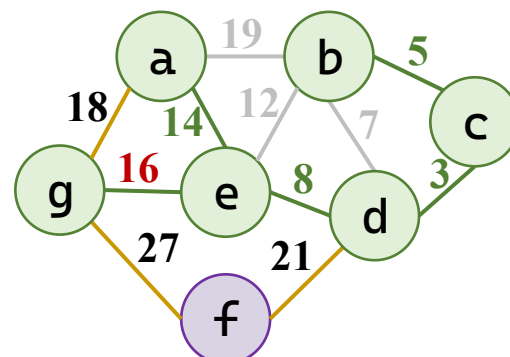
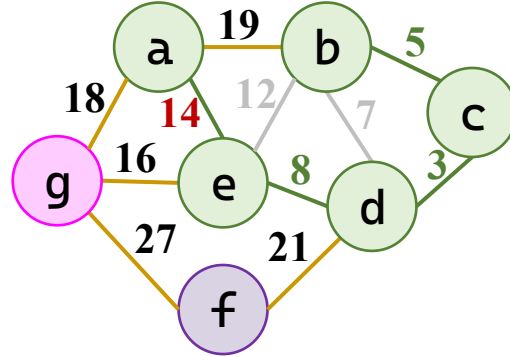
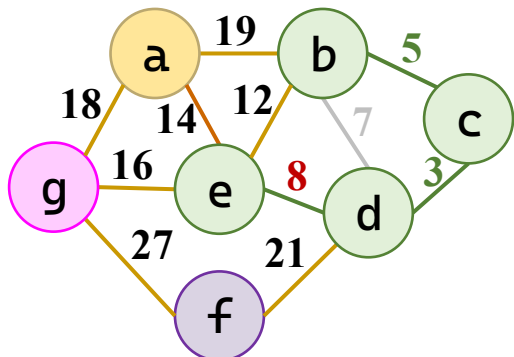
if (j == -1) {
    printf("图不连通, 无法生成最小生成树! \n");
    return FAILURE;
}
// 2. 选中边
visited[j] = 1;
mst_edges[mst_count].u = adjvex[j];
mst_edges[mst_count].v = j;
mst_edges[mst_count].w = min;
mst_count++;
total_weight += min;
// 3. 更新 lowcost 和 adjvex
for (int i = 0; i < G->vexnum; i++)
    if (!visited[i] && G->arcs[j][i].adj < lowcost[i]) {
        lowcost[i] = G->arcs[j][i].adj;
        adjvex[i] = j;
    }
}
return SUCCESS;
}

```

Kruskal算法——加边法



始	终	权
c	d	3
b	c	5
b	d	7
d	e	8
b	e	12
a	e	14
e	g	16
a	g	18
a	b	19
d	f	21
f	g	27



Kruskal算法——加边法

- 算法

- 提取所有边

```
Edge edges[MAX_VERTEX * MAX_VERTEX];
int edge_num = 0;
for (i = 0; i < G->vexnum; i++)
    for (j = i + 1; j < G->vexnum; j++)
        if (G->arcs[i][j].adj != INFINITY) {
            edges[edge_num].u = i;
            edges[edge_num].v = j;
            edges[edge_num].w = G->arcs[i][j].adj;
            edge_num++;
        }
```

- 按权重升序排列

```
for (i = 0; i < edge_num - 1; i++)
    for (j = 0; j < edge_num - 1 - i; j++)
        if (edges[j].w > edges[j + 1].w) {
            Edge tmp = edges[j];
            edges[j] = edges[j + 1];
            edges[j + 1] = tmp;
        }
```

- 初始化，每个点各自染色

```
int comp[MAX_VERTEX];
for (int i = 0; i < G->vexnum; i++)
    comp[i] = i;
```

Kruskal算法——加边法

• 算法

- 每次添一条边
- 判断是否同色成环
- 如不成环，选中
- 染色

```
for (int i = 0; i < edge_num; i++) {  
    int u = edges[i].u, v = edges[i].v;  
    if (comp[u] != comp[v]) {  
        mst_edges[mst_count] = edges[i];  
        mst_count++;  
  
        int old_id = comp[v];  
        int new_id = comp[u];  
        for (int k = 0; k < G->vexnum; k++)  
            if (comp[k] == old_id)  
                comp[k] = new_id;  
        total_weight += edges[i].w;  
        if (mst_count == G->vexnum - 1) break;  
    }  
}
```

```

void Kruskal(AdjMatrix* G) {
    // 1. 提取所有边
    Edge edges[MAX_VERTEX * MAX_VERTEX];
    int edge_num = 0;
    for (int i = 0; i < G->vexnum; i++)
        for (int j = i + 1; j < G->vexnum; j++)
            if (G->arcs[i][j].adj != INFINITY) {
                edges[edge_num].u = i;
                edges[edge_num].v = j;
                edges[edge_num].w = G->arcs[i][j].adj;
                edge_num++;
            }
    // 2. 按权值排序 (冒泡)
    for (int i = 0; i < edge_num - 1; i++)
        for (int j = 0; j < edge_num - 1 - i; j++)
            if (edges[j].w > edges[j + 1].w) {
                Edge tmp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = tmp;
            }
}

```

// 3. 初始化连通分量数组

```
int comp[MAX_VERTEX];
for (int i = 0; i < G->vexnum; i++) comp[i] = i;
Edge mst_edges[MAX_VERTEX];    // 存储选中边
int mst_count = 0, total_weight = 0;
for (int i = 0; i < edge_num; i++) {
    int u = edges[i].u, v = edges[i].v;
    if (comp[u] != comp[v]) {
        mst_edges[mst_count] = edges[i];    // 存储选中边
        mst_count++;
        int old_id = comp[v], new_id = comp[u];
        for (int k = 0; k < G->vexnum; k++)    // 合并分量
            if (comp[k] == old_id) comp[k] = new_id;
        total_weight += edges[i].w;
        if (mst_count == G->vexnum - 1) break;
    }
}
if (mst_count != G->vexnum - 1) {
    printf("图不连通，无法生成最小生成树! \n");
    return FAILURE;
}
return SUCCESS;
}
```

Kruskal算法的并查集改进

- 并查集：改为染色为记录共同的祖先

- 查找根节点

```
int find(int parent[], int x) {  
    if (parent[x] != x)  
        parent[x] = find(parent, parent[x]);  
    return parent[x];  
}
```

- 合并两个集合

```
void union_set(int parent[], int rank[], int x, int y) {  
    int rx = find(parent, x), ry = find(parent, y);  
    if (rx == ry) return;  
    if (rank[rx] < rank[ry])  
        parent[rx] = ry;  
    else if (rank[rx] > rank[ry])  
        parent[ry] = rx;  
    else {  
        parent[ry] = rx; rank[rx]++;  
    }  
}
```

Kruskal算法的并查集改进

- 并查集：改为染色为记录共同的祖先

- 并查集初始化替代
原染色数组

```
int parent[MAX_VERTEX], rank[MAX_VERTEX];  
for (int i = 0; i < G->vexnum; i++) {  
    parent[i] = i; rank[i] = 0;  
}
```

- 使用并查集的查找判断替代判断同色

```
if (find(parent, u) != find(parent, v)) {
```

- 使用并查集的合并替代染色

```
union_set(parent, rank, u, v);
```

// 查找根节点 (路径压缩)

```
int find(int parent[], int x) {  
    if (parent[x] != x)  
        parent[x] = find(parent, parent[x]);  
    return parent[x];  
}
```

// 合并两个集合 (按秩合并)

```
void union_set(int parent[], int rank[], int x, int y) {  
    int rx = find(parent, x);  
    int ry = find(parent, y);  
    if (rx == ry) return;  
    if (rank[rx] < rank[ry])  
        parent[rx] = ry;  
    else if (rank[rx] > rank[ry])  
        parent[ry] = rx;  
    else {  
        parent[ry] = rx;  
        rank[rx]++;  
    }  
}
```

```

int Kruskal_DisjointSet(AdjMatrix* G) {
    Edge edges[MAX_VERTEX * MAX_VERTEX];
    int edge_num = 0;
    for (int i = 0; i < G->vexnum; i++)
        for (int j = i + 1; j < G->vexnum; j++)
            if (G->arcs[i][j].adj != INFINITY) {
                edges[edge_num].u = i; edges[edge_num].v = j;
                edges[edge_num].w = G->arcs[i][j].adj;
                edge_num++;
            }
    for (int i = 0; i < edge_num - 1; i++)           // 冒泡排序
        for (int j = 0; j < edge_num - 1 - i; j++)
            if (edges[j].w > edges[j + 1].w) {
                Edge tmp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = tmp;
            }
    // 并查集初始化
    int parent[MAX_VERTEX], rank[MAX_VERTEX];
    for (int i = 0; i < G->vexnum; i++) {
        parent[i] = i;
        rank[i] = 0;
    }
}

```

```

Edge mst_edges[MAX_VERTEX];
int mst_count = 0, total_weight = 0;
for (int i = 0; i < edge_num; i++) {
    int u = edges[i].u, v = edges[i].v;
    if (find(parent, u) != find(parent, v)) {
        mst_edges[mst_count] = edges[i];
        mst_count++;
        union_set(parent, rank, u, v);
        total_weight += edges[i].w;
        if (mst_count == G->vexnum - 1) break;
    }
}
if (mst_count != G->vexnum - 1) {
    printf("\n图不连通，无法生成最小生成树! \n");
    return FAILURE;
}
return SUCCESS;
}

```

目录

	2	图的存储结构
	3	图的遍历
	4	最小生成树
	5	有向无环图及其应用
	6	最短路径

有向无环图

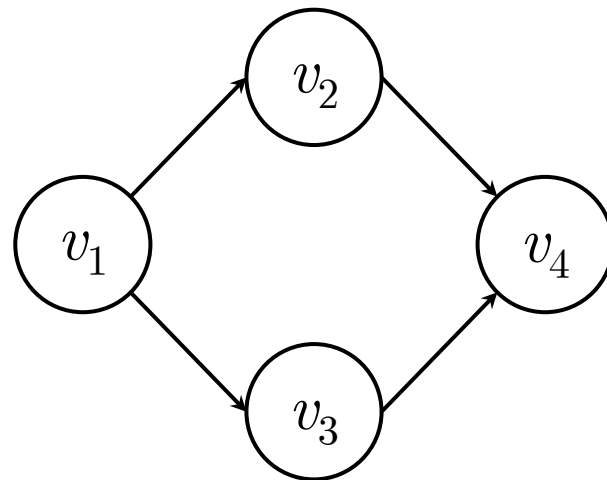
- 有向无环图 (Directed Acyclic Graph)

- 一个没有回路 (即无环) 的有向图。

- 主要用于研究工程项目的工序问题、工程时间进度问题等。

一个**工程**(project)都可分为若干个称为**活动**(active)的**子工程(或工序)**，各个子工程受到一定的条件约束：某个子工程必须开始于另一个子工程完成之后；整个工程有一个开始点(起点)和一个终点。人们关心：

- ◆ 工程能否顺利完成？（拓扑排序问题）
- ◆ 估算整个工程完成所必须的最短时间是多少？影响工程的关键活动是什么？（关键路径问题）



有向无环图

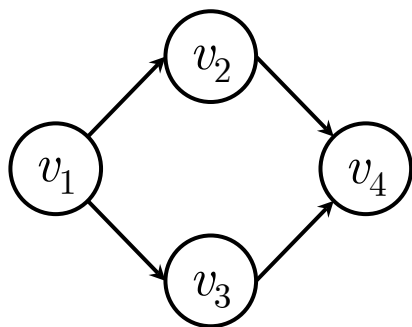
- AOV网：图中顶点表示活动，有向边表示活动之间的优先关系，这样的有向图称为顶点表示活动的网 (Activity On Vertex Network，AOV网)。
- 检查有向图中是否存在环的方法：
 - 方法一：如果有向图G的某个顶点v出发，进行深度优先搜索遍历时产生顶点u到顶点v的回边，则图G存在包含顶点u和顶点v的环。
 - 方法二：对有向图G进行拓扑排序。如果所有顶点都包含在“拓扑序列”中，则图G不存在环。

拓扑序列

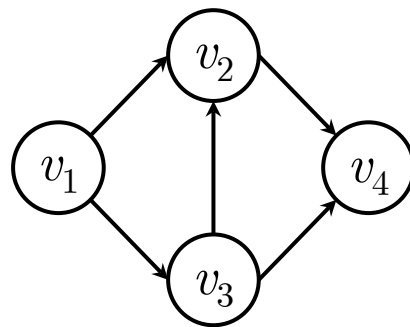
- 拓扑序列

- 按照有向图的弧 (次序关系), 将图中顶点排成一个线性序列 (对于图中没有限定次序关系的顶点, 可以人为地添加上任意的次序关系), 由此得到的顶点序列称为拓扑序列。

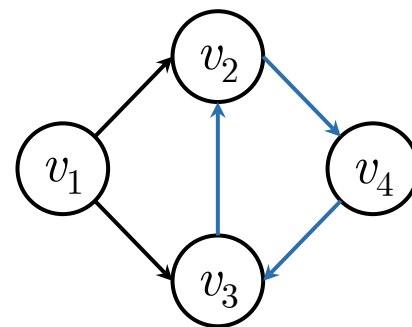
- 例如, 在由顶点 v_1 v_2 v_3 v_4 组成的有向图中, 得到拓扑序列。



v_1 v_2 v_3 v_4 和 v_1 v_3 v_2 v_4



v_1 v_3 v_2 v_4



无 (因为有环)

拓扑序列

- 拓扑排序的一般操作方法：
 - (1)从有向图中选取一个入度为0的顶点，输出它；
 - (2)从有向图中删去该顶点以及所有以它为尾的弧；
 - 重复上述两步，直至图空或者找不到入度为0的顶点为止。
- 拓扑排序算法的基本操作步骤：
 - 求各顶点的入度，将入度=0的顶点入队；
 - 当队列非空时，进行下列操作：
 - (1) 输出队首元素 v ；
 - (2) v 的所有邻接点入度减1。如果出现入度0的点，点入队。

拓扑序列的算法

• 拓扑序列的算法

- 计算各点入度
- 零入度的点入栈
- 不断处理至栈空
 - 出栈
 - 加入解
 - 删除顶点*i*及其出边，更新邻接点入度

```
FindID(G, indegree);  
  
for (i = 0; i < G->vexnum; i++)  
    if (indegree[i] == 0)  
        Push(&S, i);  
  
while (!IsEmpty(&S)) {  
    Pop(&S, &i);  
    res[count++] = i;  
    for (j = 0; j < G->vexnum; j++)  
        if (G->arcs[i][j].adj != 0) {  
            indegree[j]--;  
            if (indegree[j] == 0)  
                Push(&S, j);  
        }  
}
```

拓扑序列的算法

- 计算各点入度的算法

- 遍历各点

```
for (i = 0; i < G->vexnum; i++) {
```

- 初始化

```
indegree[i] = 0;
```

- 遍历各点

```
for (j = 0; j < G->vexnum; j++)
```

- 如果存在边，入度自增1

```
if (G->arcs[j][i].adj != 0)  
    indegree[i]++;
```

```
}
```

```

void FindID(AdjMatrix* G, int indegree[]) {
    int i, j;
    for (i = 0; i < G->vexnum; i++) {
        indegree[i] = 0;
        for (j = 0; j < G->vexnum; j++)
            if (G->arcs[j][i].adj != 0)
                indegree[i]++;
    }
}

int* Toposort(AdjMatrix* G) {
    int indegree[MAX_VERTEX];
    int i, j, count = 0;
    SeqStack S;
    int* res = (int*)malloc(sizeof(int) * G->vexnum);
    FindID(G, indegree);
    InitStack(&S);
    // 入度为0的顶点入栈
    for (i = 0; i < G->vexnum; i++)
        if (indegree[i] == 0)
            Push(&S, i);
}

```

```

while (!IsEmpty(&S)) {
    Pop(&S, &i);
    res[count++] = i;
    // 删除顶点i及其出边, 更新邻接点入度
    for (j = 0; j < G->vexnum; j++) {
        if (G->arcs[i][j].adj != 0) {
            indegree[j]--;
            if (indegree[j] == 0)
                Push(&S, j);
        }
    }
}
if (count == G->vexnum)
    return res;
else {
    free(res);
    return NULL;
}
}

```

关键路径

- AOE网 (Activity on Edge)

- AOE网是一个带权有向无环图。
- 其中，顶点表示事件，用弧表示活动，权表示活动持续的时间。
- 例如，

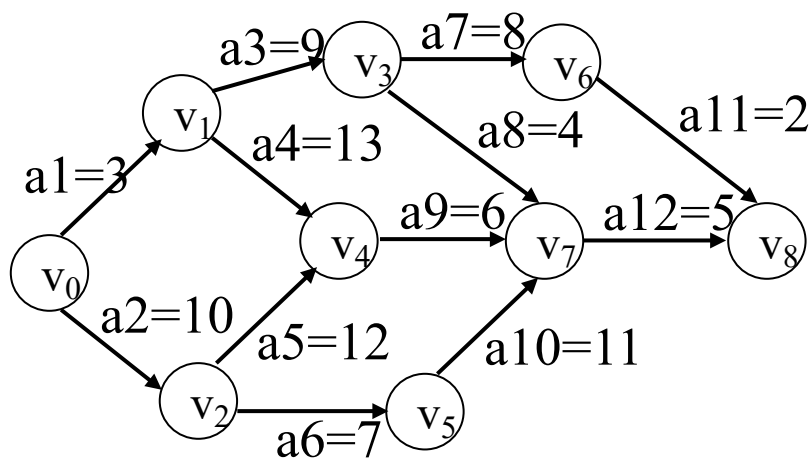


图4-1 一个AOE网

关键路径

- 问题描述：假设以AOE网表示1个施工图,权值表示完成该项子工程（或称活动）所需时间，则
 - （1）完成整个工程需要多少时间？
 - （2）哪些活动是影响工程进度的关键？
- 源点：入度=0的顶点(工程的开始点)。
- 汇点：出度=0的顶点(工程的结束点)。

关键路径

- 关键路径(工程完成最短时间)：从起点到终点的最长路径长度(路径上各活动持续时间之和)。长度最长的路径称为关键路径
- 关键活动：关键路径上的活动称为关键活动。关键活动是影响整个工程的关键。
- 事件 v_i 的最早发生时间：设 v_0 是起点，从 v_0 到 v_i 的最长路径长度称为事件 v_i 的最早发生时间，即是以 v_i 为尾的所有活动的最早发生时间。

关键路径

- 若活动 a_i 是弧 $\langle j, k \rangle$ ，持续时间是 $\text{dut}(\langle j, k \rangle)$ ，设：
 - $e(i)$ ：表示活动 a_i 的最早开始时间；
 - $l(i)$ ：在不影响进度的前提下，表示活动 a_i 的最晚开始时间；
则 $l(i) - e(i)$ 表示活动 a_i 的时间余量，
 - 若 $l(i) - e(i) = 0$ ，表示活动 a_i 是关键活动。
 - $\text{ve}(i)$ ：表示事件 v_i 的最早发生时间，即从起点到顶点 v_i 的最长路径长度；
 - $\text{vl}(i)$ ：表示事件 v_i 的最晚发生时间。
 - 则有以下关系：
$$\begin{cases} e(i) = \text{ve}(j) \\ l(i) = \text{vl}(k) - \text{dut}(\langle j, k \rangle) \end{cases} \quad (4-1)$$

关键路径

$$ve(j) = \begin{cases} 0, & j=0, \text{表示} v_j \text{是起点} \\ \max\{ve(i) + dut(<i, j>) \mid <v_i, v_j> \text{是网中的弧}\} \end{cases} \quad (4-2)$$

- 含义是：源点事件的最早发生时间设为0；除源点外，只有进入顶点 v_j 的所有弧所代表的活动全部结束后，事件 v_j 才能发生。即只有 v_j 的所有前驱事件 v_i 的最早发生时间 $ve(i)$ 计算出来后，才能计算 $ve(j)$ 。
- 方法是：对所有事件进行拓扑排序，然后依次按拓扑顺序计算每个事件的最早发生时间。

关键路径

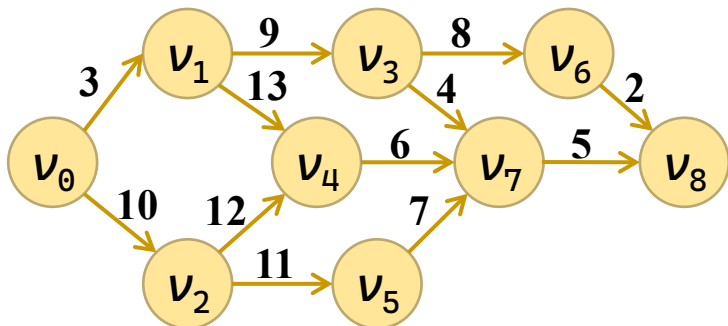
$$vl(j) = \begin{cases} ve(n-1) & j=n-1, \text{ 表示 } v_j \text{ 是终点} \\ \min\{vl(k) - dut(<j, k>) \mid <\underline{v}_j, v_k> \text{ 是网中的弧} \} \end{cases} \quad (4-3)$$

- 含义是：只有 v_j 的所有后继事件 v_k 的最晚发生时间 $vl(k)$ 计算出来后，才能计算 $vl(j)$ 。
- 方法是：按拓扑排序的逆顺序，依次计算每个事件的最晚发生时间。

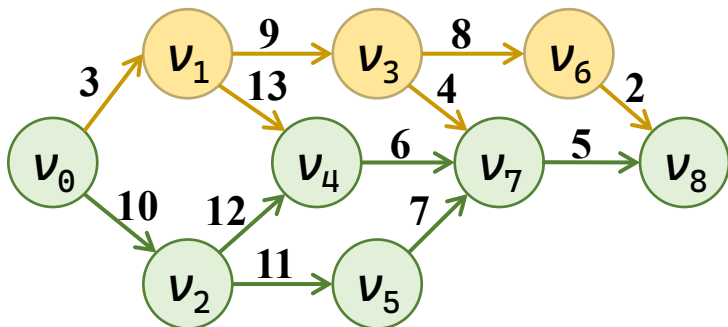
关键路径

- 求AOE中关键路径和关键活动算法思想：
 - 1、利用拓扑排序求出AOE网的一个拓扑序列；
 - 2、从拓扑排序的序列的第一个顶点(源点)开始，按拓扑顺序依次计算每个事件的最早发生时间 $ve(i)$ ；
 - 3、从拓扑排序的序列的最后一个顶点(汇点)开始，按逆拓扑顺序依次计算每个事件的最晚发生时间 $vl(i)$ ；
 - 4、根据公式4-1计算每个关键活动。

求关键路径



拓扑	0	2	5	1	4	3	7	6	8
顶点	0	1	2	3	4	5	6	7	8
ve	0	3	10	12	22	17	20	28	33
vl反	-33	-24	-23	-10	-11	-12	-2	-5	0
vl	0	9	10	23	22	21	31	28	33



起点	终点	ei	li	vl	dt	关键
0	1	0	6	9	3	
0	2	0	0	10	10	*
1	3	3	14	23	9	
1	4	3	9	22	13	
2	4	10	10	22	12	*
2	5	10	10	21	11	*
3	6	12	23	31	8	
3	7	12	24	28	4	
4	7	22	22	28	6	*
5	7	21	21	28	7	*
6	8	20	31	33	2	
7	8	28	28	33	5	*

求关键路径

• 关键路径算法

– 拓扑排序

```
topoSeq = Toposort(G);
```

– 计算最早发生时间

▪ 初始化

```
for (i = 0; i < G->vexnum; i++) ve[i] = 0;
```

▪ 对所有的下一条边

```
for (i = 0; i < G->vexnum; i++) {  
    int u = topoSeq[i];
```

```
    for (p=FirstAdjVtx(G, u); p!=ERROR; p=NextAdjVtx(G, u, p)) {
```

▪ 如果ve更短则更新

```
        if (ve[p] < ve[u] + G->arcs[u][p].adj)  
            ve[p] = ve[u] + G->arcs[u][p].adj;
```

```
    }
```

– 逆序计算最迟发生时间

▪ 初始化

```
int max_ve = ve[topoSeq[G->vexnum - 1]];  
for (i = 0; i < G->vexnum; i++) vl[i] = max_ve;
```

求关键路径

• 关键路径算法

– 逆序计算最迟发生时间

▪ 初始化

```
int max_ve = ve[topoSeq[G->vexnum - 1]];
for (i = 0; i < G->vexnum; i++) vl[i] = max_ve;
```

▪ 逆拓扑序计算

```
for (i = G->vexnum - 1; i >= 0; i--) {
    int u = topoSeq[i];
```

▪ 对所有的下一条边

```
for (p=FirstAdjVtx(G, u); p!=ERROR; p=NextAdjVtx(G, u, p)) {
```

▪ 如果vl更短则更新

```
if (vl[u] > vl[p] - G->arcs[u][p].adj)
    vl[u] = vl[p] - G->arcs[u][p].adj;
```

– 输出结果

```
}
```

▪ 如果 $ve_i = vl_p - a_{ip}$ ，则为关键路径

```
ei = ve[i];
li = vl[p] - G->arcs[i][p].adj;
tag = (ei == li) ? '*' : ' ';
```

```

int FirstAdjVtx(AdjMatrix* G, int v) {    // 获取第一个邻接点
    int i;
    for (i = 0; i < G->vexnum; i++)
        if (v != i && G->arcs[v][i].adj != INFINITY)
            return i;
    return ERROR;
}

int NextAdjVtx(AdjMatrix* G, int v, int w) { // 获取下一个邻接点
    int i;
    for (i = w + 1; i < G->vexnum; i++)
        if (v != i && G->arcs[v][i].adj != INFINITY)
            return i;
    return ERROR;
}

int CriticalPath(AdjMatrix* G) {
    int i, p, ei, li;
    char tag;
    int* ve, * vl, * topoSeq;
    ve = (int*)malloc(sizeof(int) * G->vexnum);
    vl = (int*)malloc(sizeof(int) * G->vexnum);
    if (!ve || !vl) return ERROR;

```

```

/* 拓扑排序 */
topoSeq = Toposort(G);
if (topoSeq == NULL) {
    printf("图存在环, 无法计算关键路径! \n");
    free(ve); free(vl);
    return ERROR;
}
/* 1. 按拓扑序计算 ve (最早发生时间) */
for (i = 0; i < G->vexnum; i++) ve[i] = 0;
for (i = 0; i < G->vexnum; i++) {
    int u = topoSeq[i];
    for (p=FirstAdjVtx(G, u); p!=ERROR; p=NextAdjVtx(G, u, p))
        if (ve[p] < ve[u] + G->arcs[u][p].adj)
            ve[p] = ve[u] + G->arcs[u][p].adj;
}
/* 2. 初始化 vl 为汇点的 ve (取所有 ve 的最大值更通用) */
int max_ve = ve[topoSeq[G->vexnum - 1]]; // 拓扑序尾顶点即汇点
for (i = 0; i < G->vexnum; i++) vl[i] = max_ve;
/* 3. 逆拓扑序计算 vl (最迟发生时间) */
for (i = G->vexnum - 1; i >= 0; i--) {
    int u = topoSeq[i];

```

```

    for (p=FirstAdjVtx(G, u); p!=ERROR; p=NextAdjVtx(G, u, p))
        if (vl[u] > vl[p] - G->arcs[u][p].adj)
            vl[u] = vl[p] - G->arcs[u][p].adj;
}
free(topoSeq);
/* 输出关键活动 */
printf("\n v1 v2  ei  li  vl  dt  关键\n");
printf("=====\n");
for (i = 0; i < G->vexnum; i++) {
    for (p=FirstAdjVtx(G,i); p!=ERROR; p=NextAdjVtx(G,i,p)) {
        ei = ve[i];
        li = vl[p] - G->arcs[i][p].adj;
        tag = (ei == li) ? '*' : ' ';
        printf("  %c  %c  %2d  %2d  %2d  %2d  %c\n",
            G->vertex[i], G->vertex[p], ei, li, vl[p],
            G->arcs[i][p].adj, tag);
    }
}
free(ve); free(vl);
return OK;
}

```

目录

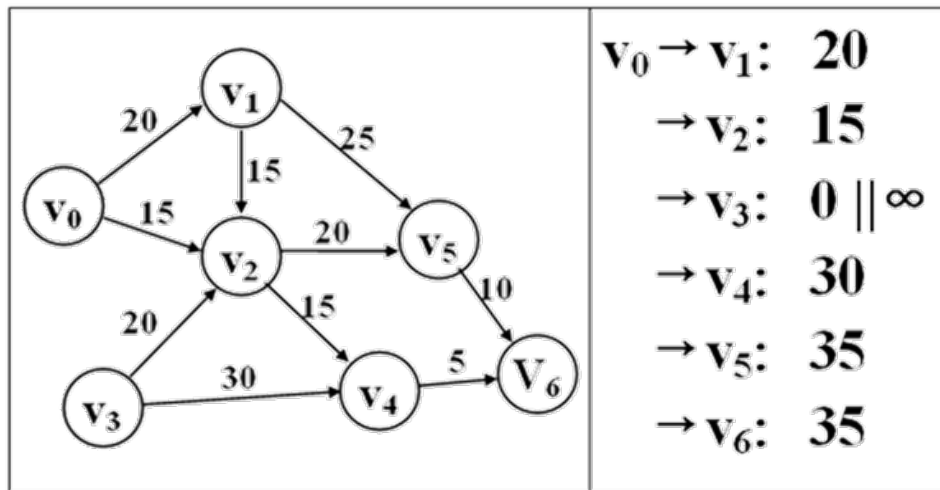
	2	图的存储结构
	3	图的遍历
	4	最小生成树
	5	有向无环图及其应用
	6	最短路径

求带权图中的最短路径问题

- 用顶点表示城市，边表示城市之间的通路，权值表示城市之间的距离。
 - 如何判断两个城市之间是否有通路？
 - 如果有通路，走哪条路最短？
- 求最短路径时，路径长度为路径上各边的权值之和。
 - 如果城市之间有通路，其权值 >0 ，否则，其权值 $=0$ (或 ∞)。
- 单源最短路径问题：
 - 设顶点 v 是指定源点，要求在有向网 G 中找到从源点 v 到其它各顶点的最短路径。

单源最短路径问题

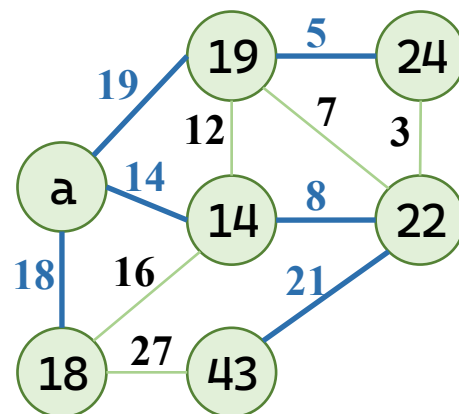
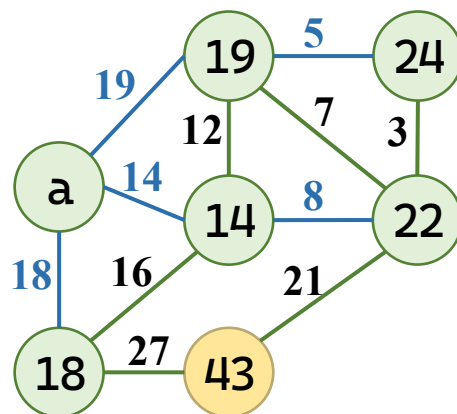
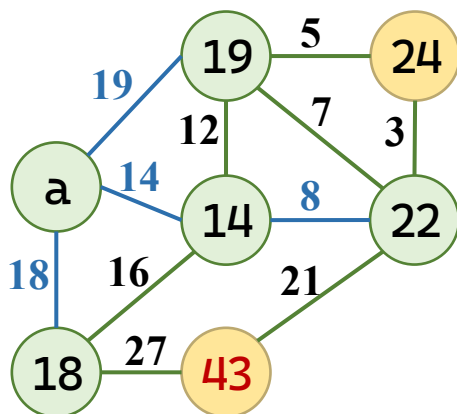
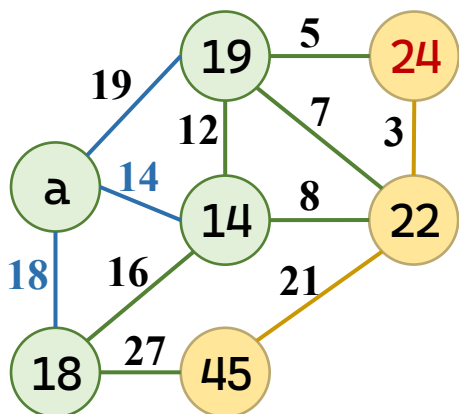
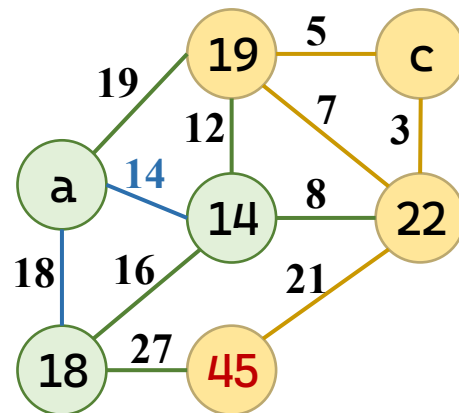
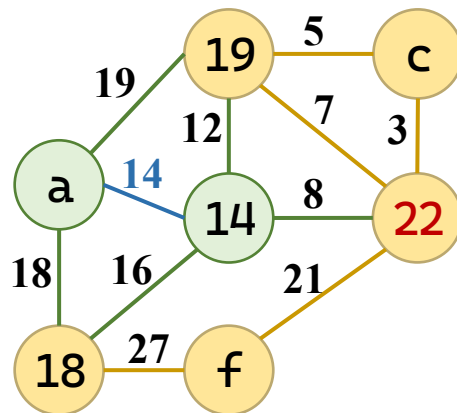
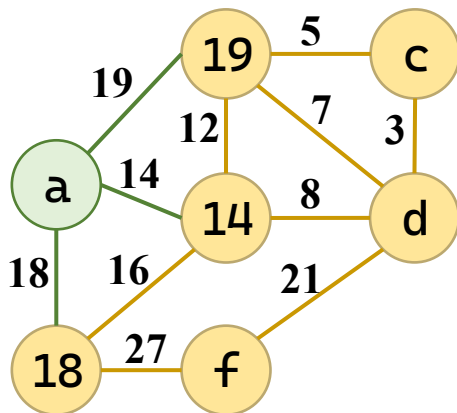
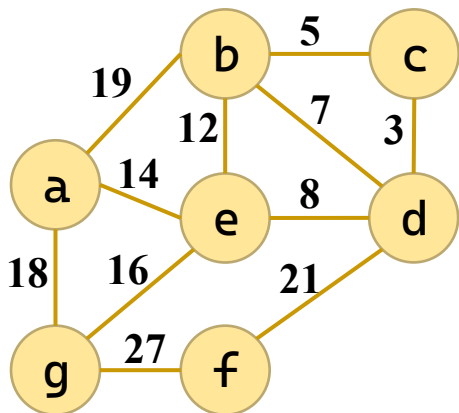
- 例：求下图中 v_0 到其它各顶点的最短路径。



狄克斯特拉 (Dijkstra) 算法

- 假设所有道路的长度都非负，如果从已确定集合S找到了s到它们的最短路径，每次从未确定集合U中找到的最小顶点u，加入S，新的集合S仍是最短路径集合。
- 反证：假设存在w,u属于U，s属于S，s...w...u比当前s...u更近，路径长度小于 $\text{dist}[u]$ 。s...w除了w以外都在S中。则，s...w的长度不小于已知的最小长度 $\text{dist}[w]$ ，s...w...u的路径长度必然不小于 $\text{dist}[w]$ ，也应该不小于当前U中最小长度 $\text{dist}[u]$ 。这一来，与u是最小长度的顶点这一规则相矛盾。

狄克斯特拉 (Dijkstra) 算法



狄克斯特拉 (Dijkstra) 算法

- 算法

- 初始化：以起始点更新各数组

```
for (i = 0; i < G->vexnum; i++) {  
    dist[i] = G->arcs[v0][i];    //  
    visited[i] = 0;              // 所有顶点均未确定最短路径  
    if (G->arcs[v0][i] != INFINITY && i != v0)  
        prev[i] = v0;           // 有直接边，前驱为源点  
    else  
        prev[i] = -1;           // 无直接边，前驱未知  
}  
// 源点到自身的距离为 0，并标记为已访问  
dist[v0] = 0; visited[v0] = 1; prev[v0] = -1;
```

狄克斯特拉 (Dijkstra) 算法

• 算法

– 循环确定最短路径

– 在U中，选择距离最小的顶点

– 标记已找到

– 更新其余未确定顶点的最短距离

```
for (i = 1; i < G->vexnum; i++) {  
    int minDist = INFINITY; u = -1;  
    for (j = 0; j < G->vexnum; j++)  
        if (!visited[j] && dist[j] < minDist) {  
            minDist = dist[j]; u = j;  
        }  
    visited[u] = 1;  
    for (j = 0; j < G->vexnum; j++) {  
        if (!visited[j] && G->arcs[u][j] != INFINITY) {  
            int newDist = dist[u] + G->arcs[u][j];  
            if (newDist < dist[j]) {  
                dist[j]=newDist; prev[j]=u; // 更新前驱  
            }  
        }  
    }  
}
```

```

void Dijkstra(AdjMatrix* G, int v0, int dist[], int prev[]) {
    int i, j, u;
    int visited[MAX_VERTEX];    // 1 表示顶点 i 已确定最短路径
    // ----- 1. 初始化 -----
    for (i = 0; i < G->vexnum; i++) {
        dist[i] = G->arcs[v0][i];    // 源点到各顶点的直接距离
        visited[i] = 0;                // 所有顶点均未确定最短路径
        if (G->arcs[v0][i] != INFINITY && i != v0)
            prev[i] = v0;                // 有直接边, 前驱为源点
        else
            prev[i] = -1;                // 无直接边, 前驱未知
    }
    // 源点到自身的距离为 0, 并标记为已访问
    dist[v0] = 0; visited[v0] = 1; prev[v0] = -1;
    // ----- 2. 主循环: 每次确定一个顶点的最短路径 -----
    for (i = 1; i < G->vexnum; i++) {
        // 2.1 在未确定最短路径的顶点中, 选择距离最小的顶点 u
        int minDist = INFINITY;
        u = -1;
    }
}

```

```

for (j = 0; j < G->vexnum; j++)
    if (!visited[j] && dist[j] < minDist) {
        minDist = dist[j];
        u = j;
    }
// 如果 u == -1, 说明剩余顶点均不可达, 提前结束
if (u == -1) break;
// 标记 u 的最短路径已确定
visited[u] = 1;
// 2.2 以 u 为中间点, 更新其余未确定顶点的最短距离
for (j = 0; j < G->vexnum; j++) {
    if (!visited[j] && G->arcs[u][j] != INFINITY) {
        int newDist = dist[u] + G->arcs[u][j];
        if (newDist < dist[j]) {
            dist[j] = newDist;
            prev[j] = u;    // 更新前驱
        }
    }
}
}
}
}
}

```

每一对顶点最短路径问题

- 问题描述

- 求图中任意一对顶点之间的最短路径。

弗洛伊德 (Floyd) 算法

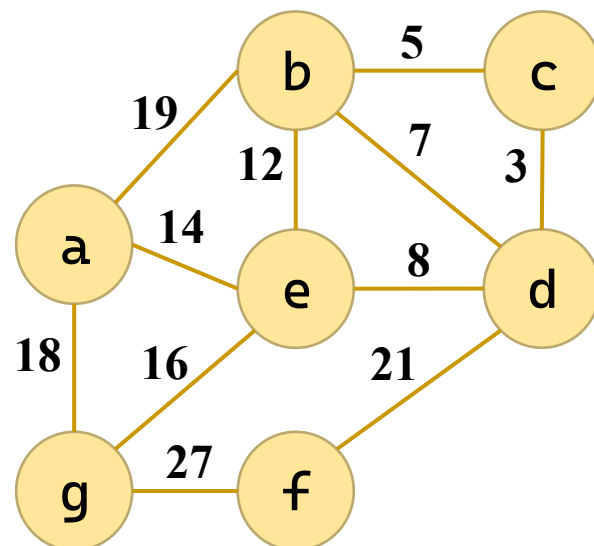
•

dist	a	b	c	d	e	f	g
a	0	19	∞	∞	14	∞	18
b	19	0	5	7	12	∞	∞
c	∞	5	0	3	∞	∞	∞
d	∞	7	3	0	8	21	∞
e	14	12	∞	8	0	∞	16
f	∞	∞	∞	21	∞	0	27
g	18	∞	∞	∞	16	27	0

path	a	b	c	d	e	f	g
a	a	a	/	/	a	/	a
b	b	b	b	b	b	/	/
c	/	c	c	c	/	/	/
d	/	d	d	d	d	d	/
e	e	e	/	e	e	/	e
f	/	/	/	f	/	f	f
g	g	/	/	/	g	g	g

更新：

1. a: bag比bg短；
2. b: abc比ac短；
abd比ad短；
cbe比ce短；
cbg比cg短；
dbg比dg短；
3. c: 无；
4. d: adf比af短；
bdf比bf短；
cde比ce短；
cdf比cf短；
edf比ef短；
5. e: aed比ad短；
aef比af短；
beg比bg短；
ceg比cg短；
deg比dg短；



弗洛伊德 (Floyd) 算法

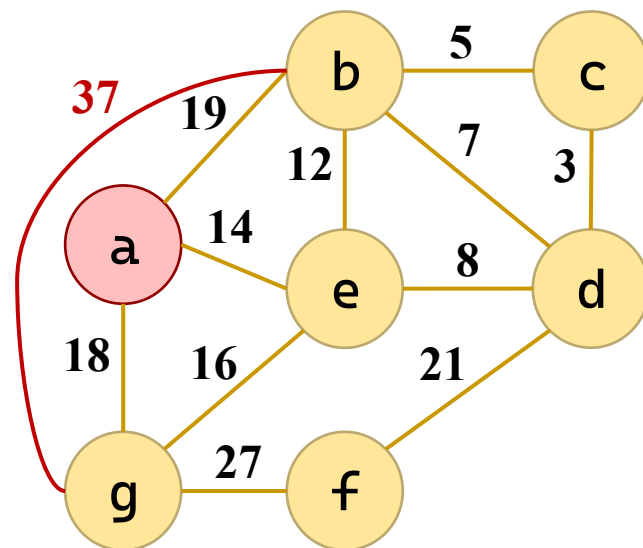
•

dist	a	b	c	d	e	f	g
a	0	19	∞	∞	14	∞	18
b	19	0	5	7	12	∞	37
c	∞	5	0	3	∞	∞	∞
d	∞	7	3	0	8	21	∞
e	14	12	∞	8	0	∞	16
f	∞	∞	∞	21	∞	0	27
g	18	∞	∞	∞	16	27	0

path	a	b	c	d	e	f	g
a	a	a	/	/	a	/	a
b	b	b	b	b	b	/	a
c	/	c	c	c	/	/	/
d	/	d	d	d	d	d	/
e	e	e	/	e	e	/	e
f	/	/	/	f	/	f	f
g	g	/	/	/	g	g	g

更新：

1. **a**: bag比bg短；
2. **b**: abc比ac短；
abd比ad短；
cbe比ce短；
cbg比cg短；
dbg比dg短；
3. **c**: 无；
4. **d**: adf比af短；
bdf比bf短；
cde比ce短；
cdf比cf短；
edf比ef短；
5. **e**: aed比ad短；
aef比af短；
beg比bg短；
ceg比cg短；
deg比dg短；



弗洛伊德 (Floyd) 算法

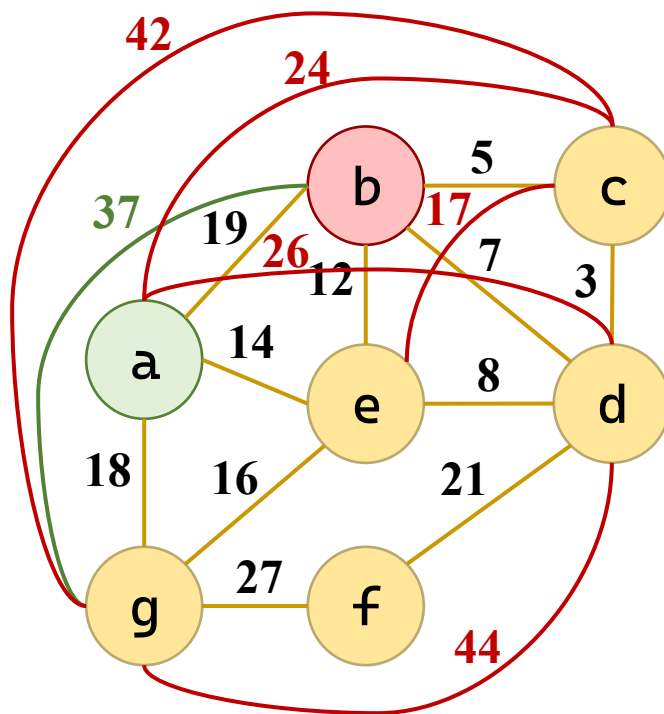
•

dist	a	b	c	d	e	f	g
a	0	19	24	26	14	∞	18
b	19	0	5	7	12	∞	37
c	∞	5	0	3	17	∞	42
d	∞	7	3	0	8	21	44
e	14	12	∞	8	0	∞	16
f	∞	∞	∞	21	∞	0	27
g	18	∞	∞	∞	16	27	0

path	a	b	c	d	e	f	g
a	a	a	b	b	a	/	a
b	b	b	b	b	b	/	a
c	/	c	c	c	b	/	a
d	/	d	d	d	d	d	a
e	e	e	/	e	e	/	e
f	/	/	/	f	/	f	f
g	g	/	/	/	g	g	g

更新：

1. a: bag比bg短；
2. b: abc比ac短；
abd比ad短；
cbe比ce短；
cbg比cg短；
dbg比dg短；
3. c: 无；
4. d: adf比af短；
bdf比bf短；
cde比ce短；
cdf比cf短；
edf比ef短；
5. e: aed比ad短；
aef比af短；
beg比bg短；
ceg比cg短；
deg比dg短；



弗洛伊德 (Floyd) 算法

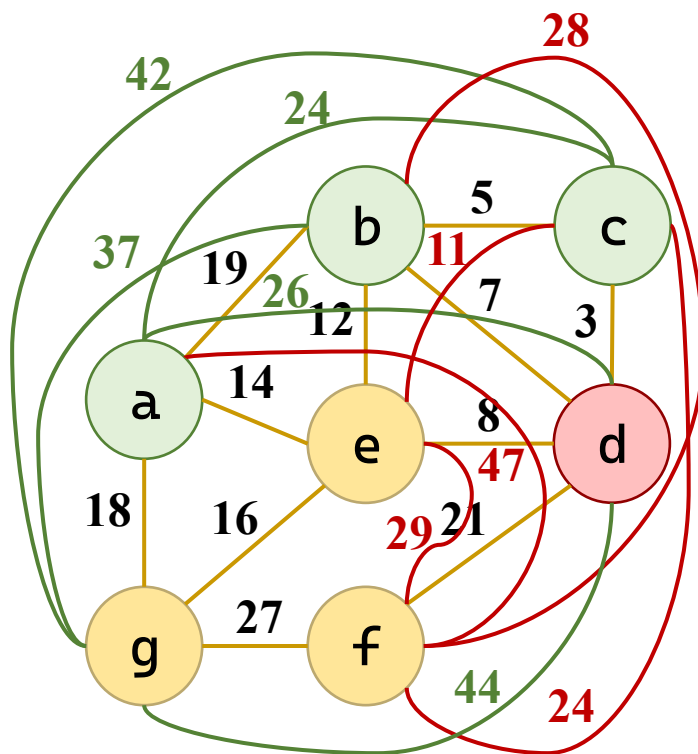
•

dist	a	b	c	d	e	f	g
a	0	19	24	26	14	47	18
b	19	0	5	7	12	28	37
c	∞	5	0	3	11	24	42
d	∞	7	3	0	8	21	44
e	14	12	∞	8	0	29	16
f	∞	∞	∞	21	∞	0	27
g	18	∞	∞	∞	16	27	0

path	a	b	c	d	e	f	g
a	a	a	b	b	a	d	a
b	b	b	b	b	b	d	a
c	/	c	c	c	d	d	a
d	/	d	d	d	d	d	a
e	e	e	/	e	e	d	e
f	/	/	/	f	/	f	f
g	g	/	/	/	g	g	g

更新：

1. a: bag比bg短；
2. b: abc比ac短；
abd比ad短；
cbe比ce短；
cbg比cg短；
dbg比dg短；
3. c: 无；
4. d: adf比af短；
bdf比bf短；
cde比ce短；
cdf比cf短；
edf比ef短；
5. e: aed比ad短；
aef比af短；
beg比bg短；
ceg比cg短；
deg比dg短；



弗洛伊德 (Floyd) 算法

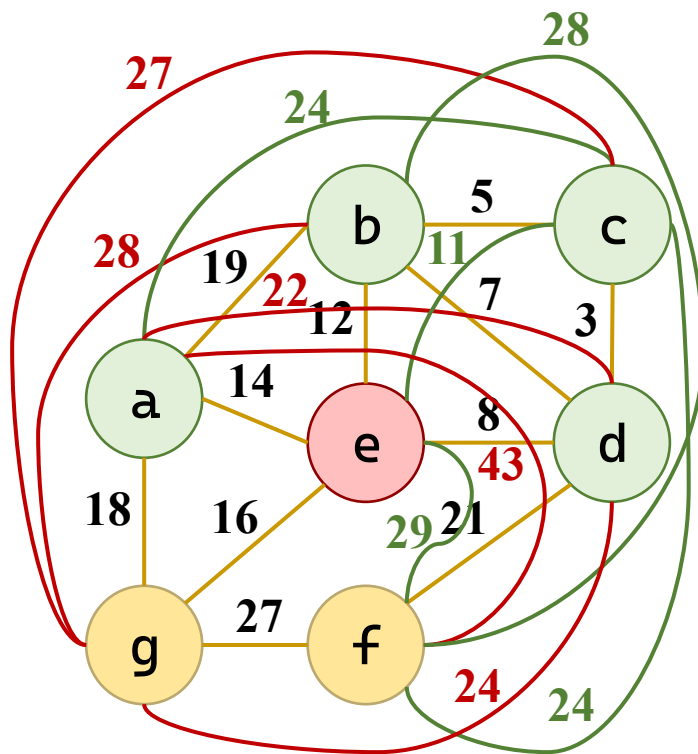
•

dist	a	b	c	d	e	f	g
a	0	19	24	22	14	43	18
b	19	0	5	7	12	28	28
c	∞	5	0	3	11	24	27
d	∞	7	3	0	8	21	24
e	14	12	∞	8	0	29	16
f	∞	∞	∞	21	∞	0	27
g	18	∞	∞	∞	16	27	0

path	a	b	c	d	e	f	g
a	a	a	b	e	a	d	a
b	b	b	b	b	b	d	e
c	/	c	c	c	d	d	e
d	/	d	d	d	d	d	e
e	e	e	/	e	e	d	e
f	/	/	/	f	/	f	f
g	g	/	/	/	g	g	g

更新：

1. a: bag比bg短；
2. b: abc比ac短；
abd比ad短；
cbe比ce短；
cbg比cg短；
dbg比dg短；
3. c: 无；
4. d: adf比af短；
bdf比bf短；
cde比ce短；
cdf比cf短；
edf比ef短；
5. e: aed比ad短；
aef比af短；
beg比bg短；
ceg比cg短；
deg比dg短；



弗洛伊德 (Floyd) 算法

起始	距离	路径	起始	距离	路径	起始	距离	路径
a→b	19	ab	c→d	3	cd	e→f	29	edf
a→c	24	abc	c→e	11	cde	e→g	16	eg
a→d	22	aed	c→f	24	cdf	f→a	43	fdea
a→e	14	ae	c→g	27	cdeg	f→b	28	fdb
a→f	43	aedf	d→a	22	dea	f→c	24	fdc
a→g	18	ag	d→b	7	db	f→d	21	fd
b→a	19	ba	d→c	3	dc	f→e	29	fde
b→c	5	bc	d→e	8	de	f→g	27	fg
b→d	7	bd	d→f	21	df	g→a	18	ga
b→e	12	be	d→g	24	deg	g→b	28	geb
b→f	28	bdf	e→a	14	ea	g→c	27	gedc
b→g	28	beg	e→b	12	eb	g→d	24	ged
c→a	24	cba	e→c	11	edc	g→e	16	ge
c→b	5	cb	e→d	8	ed	g→f	27	gf

path	a	b	c	d	e	f	g
a	a	a	b	e	a	d	a
b	b	b	b	b	b	d	e
c	/	c	c	c	d	d	e
d	/	d	d	d	d	d	e
e	e	e	/	e	e	d	e
f	/	/	/	f	/	f	f
g	g	/	/	/	g	g	g

弗洛伊德 (Floyd) 算法

• 算法

— 初始化

- 记录路径*i*→*j*的前驱
- 能到达的记*i*
无法到达记-1
- 自己的路径记*i*

```
for (i = 0; i < G->vexnum; i++) {  
    for (j = 0; j < G->vexnum; j++) {  
        d[i][j] = G->arcs[i][j].adj;  
        if (i!=j && G->arcs[i][j].adj<INFINITY)  
            p[i][j] = i;  
        else  
            p[i][j] = -1;  
    }  
    p[i][i] = i;  
}
```

— 核心三重循环

- 遍历中间点、
起点、终点
- 找更短路径更新
距离和前驱

```
for (k = 0; k < G->vexnum; k++)  
    for (i = 0; i < G->vexnum; i++)  
        for (j = 0; j < G->vexnum; j++)  
            if (d[i][k] + d[k][j] < d[i][j]) {  
                d[i][j] = d[i][k] + dist[k][j];  
                p[i][j] = p[k][j];  
            }
```

```

void Floyd(AdjMatrix* G) {
    int dist[MAX_VERTEX][MAX_VERTEX];    // 最短距离矩阵
    int path[MAX_VERTEX][MAX_VERTEX];    // 前驱矩阵: path[i][j]
    表示 i→j 路径上 j 的直接前驱
    int i, j, k;

    // ===== 1. 初始化 =====
    for (i = 0; i < G->vexnum; i++) {
        for (j = 0; j < G->vexnum; j++) {
            dist[i][j] = G->arcs[i][j].adj;
            if (i != j && G->arcs[i][j].adj < INFINITY) {
                path[i][j] = i;           // i 直接到 j, 前驱为 i
            } else {
                path[i][j] = -1;          // 不可达
            }
        }
        path[i][i] = i;                  // 自己到自己, 前驱为自己
    }

    // ===== 2. Floyd 核心三重循环 =====

```

```

for (k = 0; k < G->vexnum; k++)
    for (i = 0; i < G->vexnum; i++)
        for (j = 0; j < G->vexnum; j++)
            // 如果经过 k 点路径更短, 则更新
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
                // 更新前驱: i→j 的路径中, j 的前驱等于 k→j 路径中 j 的前驱
                path[i][j] = path[k][j];
            }
// ===== 3. 输出结果 =====
printf("\n===== Floyd 算法结果 =====\n");
printf("\n各对顶点间的最短路径: \n");
for (i = 0; i < G->vexnum; i++)
    for (j = 0; j < G->vexnum; j++)
        if (i != j && dist[i][j] < INFINITY) {
            printf("  %c → %c : 距离 = %3d, 路径 = ",
                G->vertex[i], G->vertex[j], dist[i][j]);
            printPath(i, j, path, G);
            printf("\n");
        }
printf("\n");
}

```

本节小结

- 单源最短路径
 - 从指定顶点 v ，要求在图 G 中找到从顶点 v 到其它各顶点的最短路径。
 - Dijkstra 算法时间复杂度为 $O(n^2)$ 。
- 求图 G 中任意一对顶点之间的最短路径。
 - Floyd 算法时间复杂度为 $O(n^3)$ 。

谢谢观看



廈門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang